



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Creación de *RAG*



Presentado por Lydia Blanco Ruiz
en Universidad de Burgos — 15 de mayo
de 2026

Tutor: Álgvar Arnaiz González
y César Ignacio García Osorio



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



Los doctores César Ignacio García Osorio y Álar Arnaz González, profesores del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Exponen:

Que la alumna D^a. Lydia Blanco Ruiz, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Creación de *RAG*.

Y que dicho trabajo ha sido realizado por la alumna bajo la dirección de los que suscriben, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 15 de mayo de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Álar Arnaz González

D. César Ignacio García Osorio

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	5
2.1. Objetivos específicos	5
2.2. Objetivos personales	6
3. Conceptos teóricos	7
3.1. <i>Web Scraping</i>	7
3.2. <i>LLM</i>	7
3.3. <i>RAG</i>	9
3.4. Conexión del <i>LLM</i> con el <i>RAG</i>	10
3.5. Diseño del <i>RAG</i>	11
3.6. Evaluación de la calidad del <i>RAG</i>	16
4. Técnicas y herramientas	19
4.1. Metodologías	19
4.2. Repositorio	20
4.3. Gestión de tareas	21
4.4. $\text{\LaTeX}$	21
4.5. <i>Entorno de desarrollo</i>	22
4.6. <i>Web Scraping</i>	22

4.7. Gestión de dependencias y entornos <i>Python</i>	24
4.8. Bases de datos	24
4.9. Desarrollo <i>web</i>	27
4.10. Evaluación de la calidad del código	27
4.11. Despliegue de la aplicación	28
5. Aspectos relevantes del desarrollo del proyecto	33
5.1. Inicio del proyecto	33
5.2. Web Scraping	34
5.3. <i>Retrieval Augmented Generation (RAG)</i>	35
5.4. Conversión a <i>Markdown</i>	41
5.5. Evaluación <i>RAG</i>	43
5.6. Desarrollo <i>web</i>	44
5.7. Evaluación de la calidad del código	48
5.8. Despliegue	49
5.9. Integración CI/CD	51
6. Trabajos relacionados	53
6.1. TFG relacionados	53
6.2. Proyectos relacionados	54
6.3. Análisis de las características de los proyectos	55
7. Conclusiones y Líneas de trabajo futuras	57
Bibliografía	59

Índice de figuras

3.1. Diagrama del funcionamiento del <i>RAG</i>	9
3.2. Funcionamiento de un modelo <i>LLM</i> con <i>RAG</i> y otro sin él [1]. .	11
3.3. Triada de evaluación TruEra.	17
4.1. Metodología <i>Scrum</i>	20
4.2. Relación entre las herramientas utilizadas para desplegar la aplicación <i>web</i>	29

Índice de tablas

4.1. Ventajas y desventajas de <i>Selenium</i> y <i>Playwright</i>	23
4.2. Ventajas y desventajas de <i>Qdrant</i> y <i>pgvector</i>	26
4.3. Comparativa de <i>PostgreSQL</i> y <i>SQLite</i>	29
6.1. Comparativa de las características de los proyectos.	56

1. Introducción

La contratación pública [6] constituye uno de los pilares fundamentales del sector público. A través de este proceso, las administraciones adquieren bienes, servicios y obras necesarias para el desarrollo de sus competencias, garantizando los principios de transparencia, libre concurrencia, igualdad de trato y eficiencia en el uso de los recursos públicos. En España, este procedimiento se materializa en la publicación de licitaciones que incluyen pliegos administrativos y técnicos, resoluciones, anexos y diversa documentación normativa que regula cada contrato.

Sin embargo, aunque la información es pública y accesible, su volumen, complejidad técnica y extensión documental dificultan enormemente su creación, consulta y análisis. Los pliegos de contratación pueden superar fácilmente decenas o incluso cientos de páginas, contienen lenguaje jurídico-administrativo especializado y presentan estructuras heterogéneas. Para empresas licitadoras, profesionales del sector o incluso para ciudadanos interesados, localizar información concreta puede convertirse en una tarea lenta y costosa.

En este contexto surge la necesidad de herramientas inteligentes que permitan consultar grandes volúmenes de documentación pública de forma eficiente, precisa y contextualizada. Los modelos de lenguaje de gran tamaño (*LLM*) han demostrado un gran potencial para la generación de texto y la comprensión del lenguaje natural. Sin embargo, presentan limitaciones importantes, ya que su conocimiento es estático (fecha de corte), pueden generar respuestas incorrectas con aparente seguridad (alucinaciones) y no tienen acceso directo a información específica o actualizada, como documentos concretos de licitación.

Para solventar estas limitaciones, en este Trabajo Fin de Grado se ha desarrollado *PythIA*. Un proyecto que propone el diseño e implementación de un sistema basado en la técnica *Retrieval-Augmented Generation* (*RAG*), cuyo objetivo es combinar la capacidad generativa de un *LLM* con un sistema de recuperación semántica de información sobre una colección documental formada por licitaciones públicas. El sistema permite indexar automáticamente los documentos, fragmentarlos en unidades manejables (*chunks*), representarlos mediante vectores lineales (*embeddings*) y almacenarlos en una base de datos vectorial especializada (*Qdrant*). Ante una consulta del usuario, el sistema recupera los fragmentos más relevantes y los incorpora al *prompt* del modelo generativo, mejorando la precisión, actualidad y fiabilidad de la respuesta.

El proyecto no se limita al diseño conceptual del sistema *RAG*, sino que aborda su implementación completa en un entorno realista y desplegable. Para ello, se ha desarrollado una aplicación *web* con *Flask* que permite la interacción del usuario con el sistema de forma transparente, se integra una base de datos relacional para la gestión de usuarios y consultas, y se utiliza una base de datos vectorial (*Qdrant*) para la recuperación semántica. Además, se emplea *Docker* para garantizar la reproducibilidad del entorno y facilitar el despliegue de la aplicación en producción.

Este documento es la memoria principal del proyecto. En él se describe el contexto en el que surge el proyecto, y los objetivos que se persiguen conseguir. Se incluye una explicación de los fundamentos teóricos necesarios para comprender el proyecto, y de las herramientas empleadas para su desarrollo indicando el valor que aportan al proyecto frente a otras alternativas del mercado. También, se exponen los aspectos más relevantes que tuvieron lugar durante el desarrollo explicando sus dificultades y como se solventaron. Para poner en valor real el proyecto es importante conocer el estado del arte en el campo de los modelos de lenguaje alimentados con *RAG*, explicando y comparando el proyecto con otros similares ya existentes. Finalmente, se detallan las conclusiones obtenidas tras finalizar el proyecto y se proponen vías de mejoras para futuros desarrollos.

Junto con la memoria se entrega los siguientes materiales adjuntos:

- Documentación técnica: detallada en los anexos. En ella se describe la planificación temporal del proyecto, explicando la planificación de cada *sprint* y comparándola con lo que se realizó en realidad. Se realiza un estudio sobre la viabilidad del proyecto teniendo en cuenta tanto si es económicamente rentable como su viabilidad legal. Se

especifican tanto los requisitos, funcionales y no funcionales explicando de manera detallada los casos de uso del proyecto, como el diseño de las distintas partes de la aplicación (arquitectura, datos, procedimientos e interfaces). En este documento, también, se incluyen los manuales del programador y del usuario. Finalmente, se realiza un estudio sobre la sostenibilidad del proyecto.

- *PythIA*: página *web* del proyecto. Accesible a través de la dirección de dominio: <https://pythia.es/>
- Repositorio *GitHub*¹ del proyecto. Donde se encuentra el código, la documentación y el desarrollo del proyecto. Pudiendo verse en él la planificación, ya que cada *milestone* se corresponde con un *sprint*.
- Progreso del proyecto: ha sido gestionado utilizando *ZenHub*² mediante la utilización de un tablero *kanban*, los *story points* y de los *Milestone Burndown*. Este proyecto asociado al de *GitHub*.
El progreso del proyecto también puede verse en un *GitHub Project*³ asociado al repositorio de *GitHub*, en el cual puede observarse una tabla con las *issues* de cada *sprint* y un tablero *kanban*. Además en el apartado *insights*⁴ se han generado los *Burndown chart* de cada *sprint*.
- Informe de calidad: para analizar la calidad del código de la aplicación definitiva (la que se encuentra dentro de la carpeta *PythIA* en *GitHub*) se ha utilizado *SonarCloud*⁵.

¹https://github.com/lbr1014/TFG_RAG.git

²[https://app.zenhub.com/workspaces/tfgrag-68d94a186434450034791a44/
board](https://app.zenhub.com/workspaces/tfgrag-68d94a186434450034791a44/board)

³<https://github.com/users/lbr1014/projects/1>

⁴<https://github.com/users/lbr1014/projects/1/insights>

⁵https://sonarcloud.io/project/overview?id=lbr1014_TFG_RAG

2. Objetivos del proyecto

El objetivo principal del proyecto es diseñar e implementar un sistema de *Retrieval-Augmented Generation* (*RAG*) que permita enriquecer las respuestas de un modelo de lenguaje mediante la recuperación y utilización de información contextual proveniente de una colección documental.

2.1. Objetivos específicos

Los objetivos identificados en el presente trabajo fin de grado son los siguientes:

- Recopilar la colección documental.
 - Automatizar la obtención de los documentos (licitaciones del estado) a través de *Web Scraping*.
- Preprocesar la colección documental.
 - Convertir la colección documental (recopilada en PDF) a Markdown.
 - Normalizar y limpiar el texto.
 - Segmentar los documentos en fragmentos (*chunks*).
 - Generar *embeddings* para representar semánticamente los fragmentos.
- Implementar y gestionar un índice vectorial.
 - Almacenar los *embeddings* en una base de datos vectorial.
 - Optimizar consultas de similitud para recuperar contexto relevante.

- Desarrollar la capa de interacción con el usuario
 - Transformar la consulta en su representación vectorial.
 - Recuperar los fragmentos más relevantes de la base de datos vectorial.
 - Utilizar los fragmentos recuperados para enriquecer el *prompt* del usuario y así construir el *prompt* aumentado que finalmente se pasa al *Large Language Model (LLM)*.
- Desarrollar interfaz que facilite al usuario la interacción con el modelo para que el proceso de aumentado del *prompt* le sea transparente.
 - Desarrollar una aplicación con *Flask* para que el usuario pueda interactuar con el modelo.
 - Levantar la aplicación *Flask* con *Docker* para asegurar el correcto funcionamiento de la aplicación en cualquier entorno.
 - Desplegar la aplicación *Flask* en un entorno preparado para la producción, aplicando *Gunicorn* y *Nginx*, garantizando la seguridad mediante un certificado SSL/TLS.
 - Asignar un dominio DNS a la aplicación para facilitar el acceso.

2.2. Objetivos personales

- Adquirir conocimientos sobre los sistemas *RAG*, comprendiendo sus fundamentos, componentes y aplicaciones en el ámbito de la inteligencia artificial.
- Profundizar en el estudio de los modelos de lenguaje de gran tamaño (*LLM*).
- Desarrollar habilidades prácticas en la implementación de sistemas basados en *RAG* y *LLM*.
- Aprender a aplicar los sistemas *RAG* y los *LLM* en la resolución de problemas reales.
- Aplicar los conocimientos adquiridos durante la carrera.

3. Conceptos teóricos

En este apartado se van a definir los conceptos más relevantes a tener en cuenta sobre el proyecto, por lo cual se considera importante definir qué es el *Web Scraping*, qué es un *RAG* y cuáles son sus aportaciones más significativas a los *LLM* que ya existen. Además, se considera importante contextualizar su origen, y para ello se va a comenzar explicando que son los *LLM* y sus principales limitaciones.

3.1. *Web Scraping*

Web Scraping es el proceso automático de extraer información expuesta en páginas web y transformarla en datos estructurados. Se realiza mediante programas que descargan y analizan el código HTML o emulan la navegación de un usuario con un navegador controlado por código. Se apoya en técnicas de rastreo (*crawling*), *parseo* y normalización para preparar el dato para su análisis o integración en sistemas [22].

Existe una convención estándar, llamada *Robots Exclusion Protocol* (`robots.txt`), mediante la cual un servidor web indica qué *URLs* se deben evitar rastrear. En este archivo se especifica qué partes del sistema están permitidas o prohibidas para cada tipo de *bot*. Aunque, el incumplimiento de este protocolo no tiene consecuencias legales es éticamente correcto cumplirlo [15].

3.2. *LLM*

Los *Large Language Model* (*LLM*), que se traduce como los grandes modelos del lenguaje, son modelos de aprendizaje automático que han

sido entrenados con grandes conjuntos de datos generados por humanos. Estos datos se usan para encontrar patrones estadísticos y replicar nuestras habilidades lingüísticas. Por lo cual, se puede decir que en esencia son modelos de predicción del siguiente *token* o lo que es lo mismo de la siguiente palabra [14].

Algunos ejemplos de *LLM* son **ChatGPT (OpenAI)**⁶, **Claude (Anthropic)**⁷ o **Llama (Meta AI)**⁸.

Las principales características de los *LLM* son las siguientes:

- Naturaleza de predicción del próximo *token*: como se ha mencionado anteriormente los *LLM* seleccionan la palabra más probable de una distribución. La elección de esta palabra se basa en los patrones estadísticos que ha aprendido con los datos del entrenamiento.
- Memoria paramétrica: es el conocimiento interno del *LLM*. Consiste en la información con la que ha sido entrenado y se basa únicamente en sus parámetros. Por lo cual es una memoria limitada y estática.

Las principales limitaciones de los *LLM* son las siguientes:

- Desactualizado: los *LLM* solo pueden acceder a la información presente en sus datos de entrenamiento, que tienen una fecha límite específica conocida como «fecha de corte del conocimiento». Cualquier evento, descubrimiento o información posterior a esta fecha no estará disponible para el modelo, lo que puede generar respuestas desactualizadas. Además, el entrenamiento de un *LLM* es muy costoso y requiere mucho tiempo, por lo que estos modelos no se reentrenan con frecuencia, perpetuando esta limitación temporal.
- Alucinaciones: a veces los *LLM* dan respuestas incorrectas con gran confianza de manera que parece que la información que te está proporcionando es verdadera. Esto se debe a su naturaleza de predicción del próximo *token*.
- Limitación de conocimiento: los *LLM* se entrenan con datos públicos, lo que limita su conocimiento, ya que no tienen conocimientos de información que no sea pública, como, por ejemplo, documentos internos de una empresa, información de los clientes o de los productos.

⁶<https://chat.openai.com>

⁷<https://claude.ai>

⁸<https://ai.meta.com/llama/>

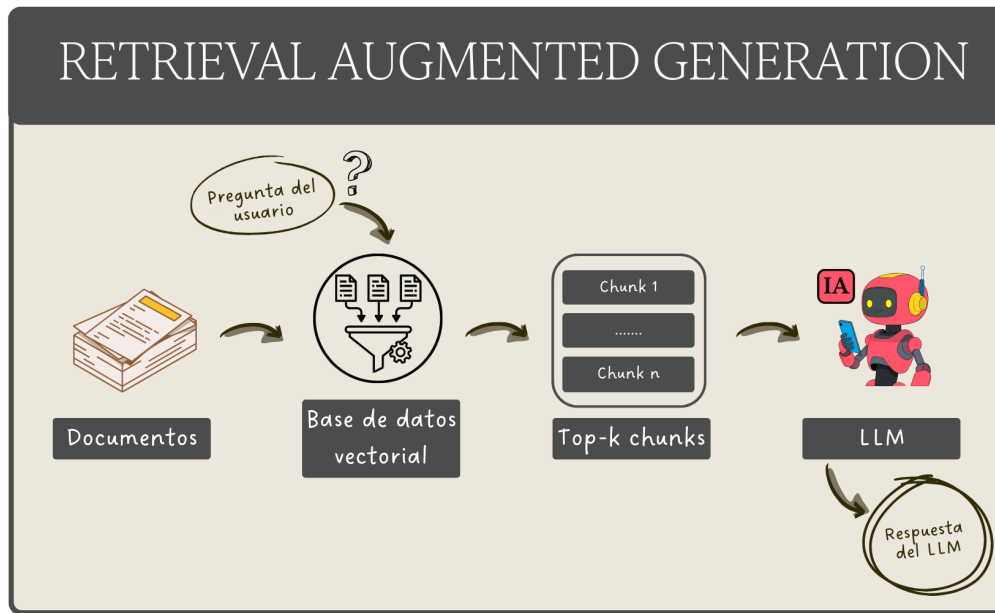


Figura 3.1: Diagrama del funcionamiento del *RAG*.

Para solventar las limitaciones que se han comentado, la solución es darle al *LLM* el contexto que le falta. El contexto del *LLM* es la información no paramétrica que se le proporciona en la consulta o *prompt* para que el modelo genere una respuesta más precisa, actual y fundamentada. Para proporcionarle este contexto se puede expandir la memoria del *LLM* mediante generación aumentada, es decir, incorporando un *RAG*.

3.3. *RAG*

Retrieval Augmented Generation (RAG), o en español, generación aumentada por recuperación, es una técnica que consiste en recuperar la información que sea relevante de una fuente externa. Con dicha información se aumenta la entrada al *LLM*, lo que permite que el *LLM* genere una respuesta mucho más precisa, contextual y confiable (ver pasos en la imagen 3.1). Para realizar esto, el *RAG* usa tres pasos: recuperar, aumentar y generar.

El *RAG* combina dos tipos de memoria:

- La memoria paramétrica: es la que típicamente tienen los *LLM*, que como se ha visto es limitada al conocimiento disponible en el momento del entrenamiento y estática.
- La memoria no paramétrica: es información externa al *LLM* que se le puede proporcionar. Es un conocimiento externo y dinámico que se puede extender tanto como se desee y que puede contener documentos propietarios.

Por lo cual conociendo esto, se puede decir que la generación aumentada por recuperación (*RAG*) es una metodología que busca ampliar la memoria paramétrica de un *LLM* incorporando una memoria no paramétrica explícita. A través de un recuperador, se obtiene información relevante de esta memoria externa, la cual se añade al *prompt* y luego se envía al *LLM*. De esta forma, el modelo puede producir respuestas más contextuales, confiables y precisas [14].

Los objetivos de *RAG* son:

- Hacer que los *LLM* respondan con información actualizada.
- Hacer que los *LLM* respondan información precisa, contextual y confiable.
- Hacer que los *LLM* conozcan información propietaria.

Las principales ventajas de *RAG* son:

- Conocimiento del contexto profundo: esto se debe a que la información adicional ayuda al *LLM* a generar respuestas contextualmente apropiadas, ya que, las respuestas se basan en datos específicos. Esto aumenta la confianza del usuario.
- Citar fuentes: como la información se extrae de fuentes conocidas las puede citar en su respuesta. Esto dispara la fiabilidad y permite a los usuarios comprobar la información obtenida.
- Reduce las alucinaciones: el sistema esta anclado a los hechos, por lo que se reducen las respuestas inexactas o falsas.

3.4. Conexión del *LLM* con el *RAG*

El primer paso es el de recopilar los documentos externos y convertirlos a vectores numéricos (*embeddings*) y almacenarlos. Cuando llega una consulta (*prompt*), en lugar de buscar coincidencias por palabras, se busca en ese

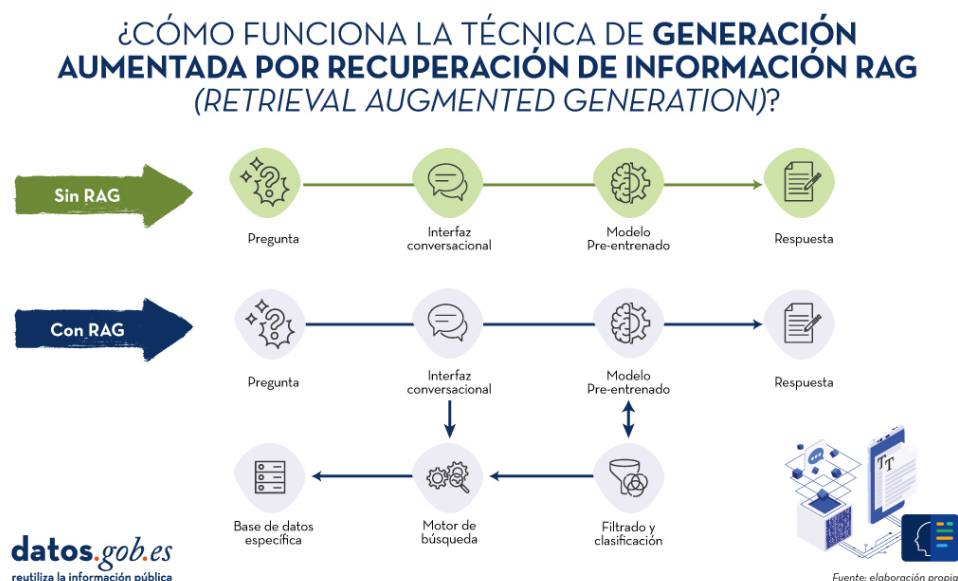


Figura 3.2: Funcionamiento de un modelo *LLM* con *RAG* y otro sin él [1].

espacio vectorial los fragmentos que son semánticamente más parecidos. A esto se le conoce como búsqueda por significado. Esos fragmentos se incorporan como contexto al *LLM*, de modo que el *RAG* combina la capacidad de razonamiento del modelo con una recuperación de información altamente precisa. En la imagen 3.2 se puede ver la diferencia de funcionamiento entre un modelo que emplea *RAG* y uno que no lo hace [1].

3.5. Diseño del *RAG*

Un sistema *RAG* se construye en dos *pipelines* complementarios, el de indexación (*offline*/asíncrono), y el de generación (*online*/ tiempo real). El primero crea y mantiene la memoria no paramétrica o base de conocimiento. El segundo gestiona la interacción con el usuario, realizando la recuperación, el aumento del *prompt* y la generación de la respuesta. Esta separación es estructural en el diseño de *RAG* y permite optimizar la precisión y la latencia del sistema [14].

***Pipeline* de indexación (*offline*)**

Su objetivo es consolidar y preparar el conocimiento para que el *pipeline* de generación pueda consultarlo eficientemente. Consta de cinco pasos:

1. Conectarse a las fuentes externas.
2. Extraer y *parsear* los documentos.
3. Dividir textos largos en fragmentos más pequeños denominados *chunks*.
4. Convertir los *chunks* a un formato adecuado.
5. Almacenar en una base de datos vectorial los *embeddings*.

Además de crearlo, este *pipeline* mantiene y actualiza el conocimiento base, por lo que debe ejecutarse de forma periódica [14].

Los componentes clave del *pipeline* de indexación son:

- Carga de datos (*data loading*): consiste en conectarse a las fuentes, por ejemplo, sistemas de ficheros, *data lakers*, CMS o APIs, para la extracción del texto en bruto y el parsing de este en formatos heterogéneos, como pueden ser PDF, DOCX, JSON o HTML. Luego se le añaden metadatos y finalmente se hace limpieza quitando código, etiquetas raras y enmascarando datos confidenciales.
- División de texto (*chunking*): segmentación en unidades manejables, los *chunks*, para mejorar la recuperación y el alineamiento con las capacidades de contexto *LLM*. Ayuda a respetar la ventana de contexto, mitiga el problema de pérdida en el medio⁹ y facilita la búsqueda eficiente. Para dividir el texto puede usar diversos métodos:
 - Tamaño fijo: divide el texto en longitudes uniformes, como un número de caracteres o *tokens*.
 - Especializados: divide el texto basándose en su estructura como encabezados o párrafos.
 - Semántico: divide el texto basándose en su similitud de significado usando *embeddings*, aunque este método aun es experimental.

La estrategia elegida dependerá del tipo de contenido, la longitud y complejidad de la consulta, el caso de uso y el modelo de *embeddings*.

- Conversión a vectores (*embeddings*): transforma el texto a representaciones numéricas de n dimensiones. Para conocer si los *chunks* se parecen a la consulta, el sistema mide la distancia entre sus vectores,

⁹El problema de la pérdida en el medio ocurre cuando los LLM pierden precisión al procesar mucha información, priorizando el inicio y el final [31].

ya que la proximidad refleja similitud semántica. Hay diversas técnicas para medir la distancia entre los vectores, una de las más utilizadas es la similitud coseno que consiste en que si el ángulo entre dos vectores es muy pequeño su similitud es casi uno, pero hay otras como la distancia euclidiana. Hay modelos preentrenados abiertos y propietarios, la elección dependerá de su uso, rendimiento, recuperación y coste.

- Almacenamiento: los *embeddings* se almacenan en bases de datos vectoriales diseñadas para vectores de altas dimensiones. Estas bases de datos vectoriales permiten la búsqueda eficiente sobre *embeddings* y metadatos. Hay diversas opciones:
 - Índices vectoriales: son bibliotecas mínimas centradas en indexación y búsqueda de alta dimensión, no gestionan datos ni ofrecen consultas ricas o interfaces de bases de datos. Ejemplos: [FAISS¹⁰](#), [NMSLIB¹¹](#), [ANNOY¹²](#), [ScaNN¹³](#). Son útiles cuando se desea máximo control y rendimiento sin sobrecarga de base de datos.
 - Bases de datos vectoriales especializadas: añaden a lo anterior gestión de datos, seguridad, escalado, extensibilidad y metadatos, manteniendo búsqueda/similitud vectorial eficiente. Ejemplos: [Pinecone¹⁴](#), [ChromaDB¹⁵](#), [Milvus¹⁶](#), [Qdrant¹⁷](#).
 - Plataformas de búsqueda: son motores de texto tradicional ([Solr¹⁸](#), [Elasticsearch¹⁹](#), [OpenSearch²⁰](#), [Lucene²¹](#)) que han incorporado similitud vectorial a sus capacidades.
 - Motores SQL/NoSQL con capacidad vectorial: son bases de datos ya existentes que añaden tipos y operadores vectoriales. Ejemplos: [Azure SQL²²](#), [PostgreSQL/pgvector²³](#) o en NoSQL [Mon-](#)

¹⁰<https://faiss.ai/index.html>

¹¹<https://nmslib.github.io/nmslib/>

¹²<https://github.com/spotify/annoy>

¹³<https://milvus.io/docs/es/scann.md>

¹⁴<https://www.pinecone.io/>

¹⁵<https://www.trychroma.com/>

¹⁶<https://milvus.io/es>

¹⁷<https://qdrant.tech/>

¹⁸<https://solr.apache.org/>

¹⁹<https://www.elastic.co/es/enterprise-search/vector-search>

²⁰<https://opensearch.org/platform/vector-search/>

²¹<https://lucene.apache.org/core/>

²²<https://azure.microsoft.com/es-es/products/azure-sql/database>

²³<https://www.postgresql.org/about/news/pgvector-070-released-2852/>

goDB²⁴. Son útiles para consolidar datos, pero son menos especialización que una vector DB dedicada.

- Bases de datos gráficas con funciones vectoriales: grafos como Neo4j²⁵ que añaden búsqueda vectorial. Permiten combinar relaciones explícitas (grafos) con similitud semántica (vectores). Ejemplos: *path finding* sujeto a similitud.

Cuando la recuperación se externaliza, es decir, cuando la búsqueda y obtención de información no la realiza el sistema sobre su propia base vectorial, sino que se delega en un tercero (como por ejemplo una API) o en el documento que aporta el usuario, no es imprescindible construir un *pipeline* de indexación ni mantener una base vectorial. En cambio, en un *RAG* clásico, donde el sistema recupera conocimiento de su repositorio interno, sí se recomienda diseñar el *pipeline* de indexación y mantener una base vectorial propia [14].

***Pipeline* de generación (*online*)**

Gestiona la consulta del usuario de extremo a extremo. Este *pipeline* recibe la pregunta del usuario, recupera el contexto relevante de la base de datos vectorial, los *chunks*, aumenta el *prompt* con ese contexto y genera la respuesta con el *LLM*. Se compone de tres fases: *retrieval*, *augmentation* y *generation* [14].

Los componentes clave del *pipeline* de generación son:

- *Retriever*: es el motor de búsqueda sobre la base vectorial, es decir, la base de conocimiento. Devuelve los documentos más relevantes. Es crítico para la precisión del sistema y contribuye a la latencia, su calidad condiciona el rendimiento del *RAG*. El método más frecuente son los *embeddings* gracias a su capacidad semántica.
- Gestión del *prompt* (*Augmentation*): combina la consulta y el contexto recuperado. Además, aplica estrategias de ingeniería de *prompt* para maximizar la calidad de la respuesta. Las estrategias recomendadas son:
 - *Contextual prompting*: consiste en instruir al modelo para que solo responda con el contexto proporcionado.

²⁴<https://www.mongodb.com/>

²⁵<https://neo4j.com/>

- *Controlled generation prompting*: consiste en indicarle al modelo que no conteste si el contexto no permite responder a la pregunta.
 - *Few-shot prompting*: consiste en incluir ejemplos para forzar el formato de la respuesta.
 - *Chain-of-thought*: consiste en pedir pasos intermedios cuando los razonamientos son complejos.
- Configuración *LLM (Generation)*: es la selección del modelo que va a generar la respuesta final. Pueden ser modelos:
- Originales o *fine-tuned*: los originales facilitan un arranque rápido, mientras que los *fine-tuned* mejoran la adaptación de dominio, la integración del contexto y formato de salida.
 - Abiertos o propietarios: los abiertos ofrecen control y despliegue flexible, mientras que los propietarios simplifican el uso y el mantenimiento.
 - Tamaño del modelo: los grandes tienen un razonamiento mejor y un contexto más amplio. Los pequeños tienen un coste y latencia menor.

Capas de soporte

Además de los dos *pipelines*, un sistema en producción requiere orquestación, evaluación y monitorización continua, caché semántico para reducir coste y latencia, controles de seguridad para cumplimiento normativo y seguridad frente a inyecciones en el *prompt*, envenenamiento de dato²⁶ o fugas de información. Todo ello se sostiene sobre una infraestructura de servicio y un *RAGOps stack* formado por capas críticas, esenciales y de mejora que aseguran operación, calidad y seguridad extremo a extremo [14].

1. **Capas críticas**: son las capas imprescindibles del *RAGOps stack*.
 - Capa de datos: es la encargada de crear la base de conocimiento, para ello recolecta datos desde sistemas origen, los transforma y los almacena.
 - Capa de modelos: se incluyen los modelos de embeddings, de los LLM y de las tareas. Incluye el entrenamiento y la optimización de inferencia.
 - Despliegue de modelos: pueden ser servicios gestionados, auto-hospedados o locales.

²⁶El envenenamiento de datos es un ciberataque que corrompe los conjuntos de datos de entrenamiento utilizados para la Inteligencia Artificial [16].

- Orquestación de la aplicación: se encarga de coordinar la consulta, la recuperación de datos y la generación. Incluye soporte multiagente y automatización de flujos de trabajo.
2. **Capas esenciales:** son las capas que necesita el sistema, se encargan del rendimiento, la fiabilidad y de la seguridad.
 - Capa de *prompt*: diseño, versión y evaluación de *prompts* para guiar al *LLM* y reducir las alucinaciones.
 - Capa de evaluación: se encarga de medir, de forma periódica, la relevancia del contexto, fidelidad y relevancia de la respuesta.
 - Capa de monitorización: observa el proceso del *RAG* para encontrar fallos. Para ello monitoriza la latencia y el error.
 - Seguridad y privacidad del *LLM*: es la encargada de emplear métodos para evitar el *prompt injection*. Algunos de los métodos que se emplean son validar las entradas, realizar encriptados y emplear control de accesos.
 - Capa de caché: emplea caché para reducir los costes y la latencia en consultas frecuentes.
 3. **Capas de mejora:** son capas opcionales que pueden aportar beneficios dependiendo del caso de uso. Se centran en la eficiencia, usabilidad y escalabilidad del sistema.

3.6. Evaluación de la calidad del *RAG*

Se suele hacer mediante la tríada de evaluación propuesta por TruEra [18] (ver imagen de la tríada 3.3). Estructura la calidad en tres comprobaciones entre consulta (*prompt*), contexto recuperado y respuesta. Complementariamente, se emplean métricas como relevancia, precisión y *recall*, y conjuntos *ground truth* para validación objetiva y monitorización continua en producción [14].

Las tres dimensiones de la tríada son:

- Relevancia del contexto: la información que se haya recuperado tiene que ver con la pregunta.
- *Groundedness* o fidelidad: la respuesta que da el *LLM* se basa de verdad en el contexto que le hemos dado, sin alucinaciones ni omisiones clave.
- Relevancia de la respuesta: la respuesta es útil y adecuada respecto a la intención y el contexto de la consulta original.



Figura 3.3: Triada de evaluación TruEra.

Aparte de las dimensiones de la tríada, otros aspectos relevantes que el sistema debe incorporar son:

- Robustez al ruido: el sistema debe distinguir aquellos documentos relacionados pero que sean poco útiles.
- Rechazo en negativo: el sistema no debe responder cuando no hay evidencias relevantes.
- Integración de información: el sistema debe ser capaz de sintetizar múltiples documentos que posean información relevante.
- Robustez contrafactual: el sistema tiene que detectar y evitar la información inexacta en el propio *knowledge base*.

Hay dos herramientas principales de evaluación:

- *Retrieval-Augmented Generation Assessment* (**RAGAs**²⁷) [4]: es un *framework* que genera datos sintéticos y calcula métricas de la tríada. Es útil para ciclos rápidos [26].

²⁷<https://github.com/vibrantlabsai/ragas>

- *Automated RAG Evaluation System* (**ARES**²⁸) [28]: es un *framework* con un funcionamiento similar al RAGAs, pero con mayor complejidad y cantidad de datos generados. Es más robusto.

²⁸<https://github.com/stanford-futuredata/ARES>

4. Técnicas y herramientas

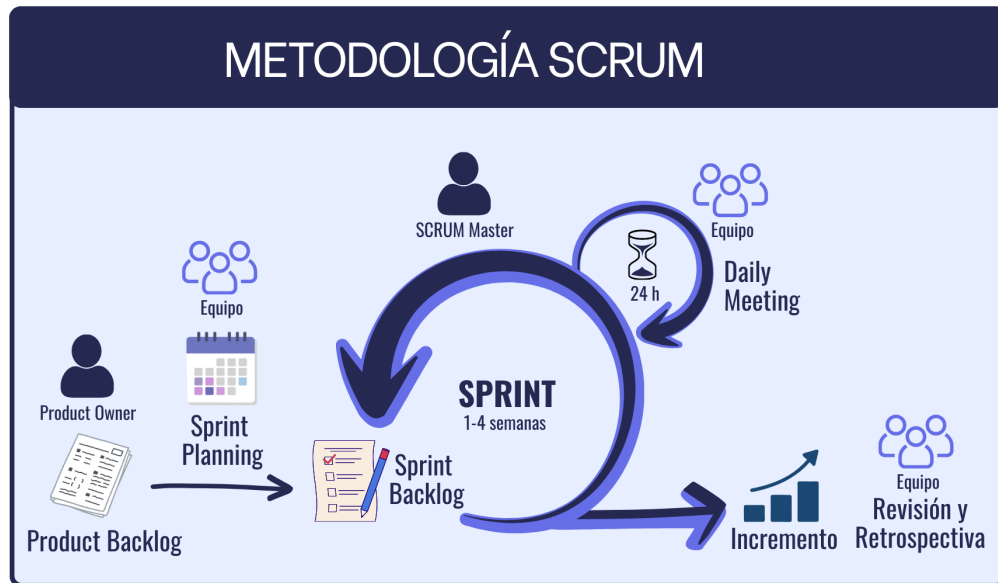
En este apartado se va a realizar una breve explicación de la metodología empleada, así como de las herramientas utilizadas para el desarrollo del proyecto. En dicha explicación se va a incluir una comparación con otras herramientas similares en la cual se muestre el motivo de la selección de dicha herramienta.

4.1. Metodologías

Para llevar a cabo el proyecto se han seguido diversas metodologías de desarrollo ágil. Estas metodologías definen la forma en la que se planifica, organiza y gestiona el desarrollo de un proyecto, permitiendo estructurar las tareas, coordinar el progreso y mejorar la calidad del resultado.

Scrum

La metodología *Scrum* es un marco de trabajo ágil orientado a la gestión y desarrollo de proyectos de manera iterativa e incremental. Se organiza en ciclos cortos denominados *sprints*, al final de los cuales se entrega un incremento funcional del producto (ver imagen 4.1 para ver el proceso *Scrum* completo). Estos *sprints* favorecen la mejora continua y la adaptación al cambio, además, aseguran una entrega temprana de valor. A diferencia de metodologías tradicionales, *Scrum* aporta mayor flexibilidad y retroalimentación constante [30].

Figura 4.1: Metodología *Scrum*.

Kanban

Para la gestión del flujo de trabajo y organización de tareas se ha adoptado la metodología *Kanban* [19]. La cual consiste en un enfoque ágil orientado a la visualización del trabajo, la optimización del flujo y la mejora continua. Se basa en la idea de que el trabajo debe gestionarse como un flujo continuo en lugar de dividirse en iteraciones cerradas. Se centra en representar el estado de las tareas de forma visual, lo que facilita la planificación, el seguimiento y la detección de cuellos de botella. Este enfoque es útil en proyectos de desarrollo *software* en los que los requisitos puedan evolucionar, haciendo necesario tener una planificación flexible.

4.2. Repositorio

Para alojar el repositorio se han valorado distintas alternativas como *Bitbucket*²⁹, *GitHub*³⁰, *GitLab*³¹ o *Trello*³². Finalmente se ha elegido *GitHub* que es una plataforma basada en la nube que permite alojar repositorios

²⁹<https://bitbucket.org/>

³⁰<https://github.com/>

³¹<https://gitlab.com/>

³²<https://trello.com/>

de código. GitHub ofrece funciones muy útiles como ramas (*branches*), *pull requests*, seguimiento de cambios (*commits*), historial del proyecto, gestión de incidencias (*issues*). Además, permite la colaboración entre usuarios, lo que es muy útil para la revisión del proyecto.

Otra ventaja es que GitHub es compatible con herramientas de integración y despliegue continuo (Github Actions), lo que facilita la automatización de pruebas y la entrega de *software*. Asimismo, permite incluir documentación directamente en el repositorio, ya sea mediante archivos `README` o a través de *wikis*, lo que mejora la claridad y la reproducibilidad del trabajo. Finalmente, al tratarse de la plataforma más utilizada a nivel mundial, cuenta con abundante documentación, lo que la convierte en una alternativa sólida y con amplio soporte [10].

4.3. Gestión de tareas

Para la gestión de las tareas del proyecto se han valorado diversas herramientas, como *ZenHub*³³, *GitHub Projects*³⁴ o *Asana*³⁵. Principalmente se ha optado por usar *ZenHub* que es una herramienta de gestión de proyectos fácilmente integrable con *GitHub*. Debido a que tiene mayor funcionalidad que *GitHub Projects*. Ambos permiten hacer tableros *Kanban* para organizar las tareas (*to-do*, *in progress*, *doing*). Pero, *ZenHub*, además, permite asignar *story points* a las tareas y generar los gráficos *burndown* de los *sprints* de manera sencilla. Sin embargo, también se ha asociado un *GitHub Project* en el cual se ha definido una tabla que organiza las tareas por *sprints* y se han guardado los *Burndown* de cada *sprint*, así como, otras gráficas de interés.

4.4. L^AT_EX

Para la edición de los documentos en L^AT_EX se ha considerado usar T_EXMaker³⁶, T_EXStudio³⁷, Visual Studio Code³⁸ y Overleaf³⁹. Finalmente se ha optado por usar T_EXMaker, ya que es un editor multiplataforma, gratuito y de código abierto. Tiene un visor PDF integrado, así como herramientas para gestionar la bibliografía mediante BibT_EX. Además, tiene

³³<https://www.zenhub.com/>

³⁴<https://github.com/>

³⁵<https://asana.com/es>

³⁶<http://www.xmlmath.net/texmaker/>

³⁷<https://www.texstudio.org/>

³⁸<https://code.visualstudio.com/>

³⁹<https://es.overleaf.com/>

una interfaz simple que facilita el aprendizaje, incorpora funciones como el autocompletado, el plegado de código y la detección de errores de compilación, sin necesidad de instalar complementos adicionales.

Como distribución L^AT_EX para procesar los documentos `.tex` se ha valorado el uso de MikTeX⁴⁰ y T_EXLive⁴¹, al final se ha elegido usar MikTeX ya que permite realizar una instalación mínima e ir añadiendo automáticamente los paquetes necesarios durante la compilación. Además, presenta un buen soporte para Windows, lo que se adapta bien a mi entorno de trabajo [33].

4.5. Entorno de desarrollo

Para el desarrollo del proyecto se ha utilizado *Visual Studio Code*⁴² [27] como entorno de desarrollo integrado (IDE) [29]. Un IDE es una aplicación que integra en un único entorno herramientas esenciales para el desarrollo de software, como editor de código, compilador, depurador y utilidades de automatización.

Visual Studio Code es un editor de código fuente multiplataforma, gratuito y muy utilizado en el ámbito profesional. Se ha seleccionado frente a otras alternativas como *PyCharm*⁴³ o entornos más pesados debido a su flexibilidad, su facilidad para incorporar extensiones y su buena integración con herramientas modernas de desarrollo como son *Git*, *Docker* y *Poetry*. Permitiendo mantener un entorno de desarrollo coherente, modular y fácilmente reproducible.

4.6. Web Scraping

Para el *web scraping* se ha valorado usar *Selenium*⁴⁴ y *Playwright*⁴⁵. Selenium controla navegadores reales y es un estándar veterano, pero Playwright es más completo. Ya que ofrece automatización multi-navegador con contextos aislados, funcionamiento headless, espera automática de elementos y APIs consistentes, lo que lo vuelve más robusto y rápido para páginas con JavaScript pesado y flujos de login. En la tabla 4.1 se muestran las ventajas y desventajas de cada uno.

⁴⁰<https://miktex.org/>

⁴¹<https://www.tug.org/texlive/>

⁴²<https://code.visualstudio.com/>

⁴³<https://www.jetbrains.com/es-es/pycharm/>

⁴⁴<https://selenium-python.readthedocs.io/>

⁴⁵<https://playwright.dev/>

Herramienta	Ventajas	Desventajas
<i>Selenium</i> [25]	Utiliza el estándar W3C <i>WebDriver</i> [23], lo que aporta interoperabilidad y soporte a largo plazo. Multilenguaje y ecosistema maduro. Permite una integración fácil. Es robusto para escalado distribuido.	Es propenso a fallos (<i>flakiness</i>) si no se dominan las esperas. La intercepción de red en menos homogénea entre navegadores.
<i>Playwright</i> [32]	Es muy estable y rápido en <i>webs</i> con <i>JavaScript</i> pesado. Presenta multitud de herramientas integradas como <i>Trace Viewer</i>, capturas de pantalla y videos. Contiene selectores potentes y permite manejar los <i>inframes</i> de forma sencilla. Tiene un control de contexto aislado muy cómodo. Su modelos asíncrono permite realizar acciones concurrentes sin bloquear el hilo empleando la <i>API async/await</i>.	No utiliza el estándar <i>WebDriver</i>.

Tabla 4.1: Ventajas y desventajas de *Selenium* y *Playwright*.

Tras probar ambos, en una fase inicial del *web scraping*, se vio que *Selenium* sigue siendo una apuesta segura, para *scraping* moderno con multiples sesiones y depuración rápida Sin embargo, *Playwright* requiere menos código y es más robusto, en su modo asíncrono gestiona de manera autónoma las esperas y los reintentos.

4.7. Gestión de dependencias y entornos *Python*

El proyecto requiere entornos aislados y reproducibles que faciliten el trabajo local, la integración continua y la generación de entregables. Se ha valorado utilizar *Poetry*⁴⁶ y *uv*⁴⁷. *Poetry* es una herramienta madura de gestión de dependencias y empaquetado que centraliza la configuración en `pyproject.toml`. Crea y gestiona entornos virtuales de forma automática, mantiene un archivo de bloqueo (`poetry.lock`) para asegurar instalaciones deterministas, permite definir grupos de dependencias, así como, generar de manera automática el `requirements.txt`. Por su parte *uv* es un gestor rápido que, al igual que *Poetry*, tiene soporte para `pyproject.toml`. Posee un buen rendimiento y una elevada velocidad gracias al uso de cachés eficientes. Finalmente, se ha optado por usar *Poetry* por su cobertura integral del ciclo de vida dentro de un flujo coherente y estable.

4.8. Bases de datos

Para el desarrollo del sistema *RAG* es necesario contar con dos tipos de bases de datos, una base de datos vectorial (ver sección 4.8.1) y una relacional (ver sección 4.8.2). La base de datos vectorial permite almacenar, indexar y buscar semánticamente en los documentos para dar respuestas precisas y rápidas. La base de datos relacional que permite gestionar la información de la aplicación (los datos de los usuarios, los documentos, las consultas, los *chunks* y los *embeddings*).

Base de datos vectorial

Una parte fundamental del desarrollo del *RAG* es la implementación de la base de datos vectorial. Se han valorado emplear diversas bases de

⁴⁶<http://python-poetry.org/docs/>

⁴⁷<https://docs.astral.sh/uv/>

datos, pero las dos que más se han estudiado son *Qdrant*⁴⁸ y *pgvector*⁴⁹. *Qdrant* es una base de datos vectorial especializada, diseñada desde cero para búsquedas de similitud de alta concurrencia y baja latencia. Presenta equilibrio entre el rendimiento, la latencia y el tiempo de indexado [13]. Por su parte, *pgvector* es una extensión de *PostgreSQL*⁵⁰ que permite almacenar embeddings en columnas vectoriales y realizar búsquedas de similitud sobre una base de datos relacional ya existente. Es útil cuando la aplicación ya se apoya en este gestor, ya que simplifica la infraestructura [35]. En la tabla 4.2 se muestran las principales ventajas y desventajas de cada opción.

Para realizar el proyecto se ha optado por utilizar *Qdrant*. Aunque *pgvector* presenta características interesantes cuando se desea mantener toda la información en una única base de datos relacional, *Qdrant* presenta un diseño especializado. Así como un mejor equilibrio entre rendimiento y latencia. Además, *Qdrant* se utiliza en proyectos reales lo que lo convierte en una muy buena opción para implementar el sistema *RAG*.

Base de datos relacional

Para la implementación de la base de datos relacional se ha valorado usar *PostgreSQL*⁵¹ y *SQLite*⁵². *PostgreSQL* es un sistema de gestión de bases de datos relacionales de código abierto, orientado a objetos con gran robustez transaccional. Por su parte *SQLite* es un sistema ligero y embebido que no funciona como servidor independiente, sino que almacena la base de datos en un fichero local. En la tabla 4.3 se muestra una comparativa entre ambos sistemas de gestión de bases de datos relacionales.

Inicialmente se comenzó integrando *SQLite*, ya que, se considero suficiente para un prototipo local del sistema *RAG*. Pero, finalmente se cambió a *PostgreSQL* porque en el proyecto se busca conseguir una arquitectura realista y desplegable que permita la concurrencia real de usuarios en el sistema.

⁴⁸<https://qdrant.tech/>

⁴⁹<https://www.datacamp.com/es/tutorial/pgvector-tutorial>

⁵⁰<https://www.postgresql.org/>

⁵¹<https://www.postgresql.org/>

⁵²<https://sqlite.org/>

Herramienta	Ventajas	Desventajas
<i>Qdrant</i> [25]	Diseñada específicamente como base de datos vectorial, con índices optimizados para búsquedas de similitud y buen compromiso entre RPS, latencia y tiempo de indexado [13].	Introduce un nuevo servicio en la arquitectura: es necesario administrar una base de datos adicional a la relacional.
	Soporta filtrado avanzado por metadatos y búsquedas híbridas (vectoriales más filtros), lo que facilita consultas complejas en <i>RAG</i> .	Requiere aprender herramientas y mecanismos de operación específicos (copias de seguridad, monitorización, actualización de índices, etc.).
	Ofrece despliegue sencillo mediante contenedores y opciones gestionadas, con APIs HTTP y RPC bien documentadas. Integración directa con bibliotecas y orquestadores de <i>RAG</i> , lo que reduce el código necesario para conectarla con el resto del sistema.	Para casos de uso pequeños puede resultar más compleja que reutilizar la base de datos transaccional existente.
<i>pgvector</i> [32]	Permite reutilizar una instancia de <i>PostgreSQL</i> ya desplegada, unificando datos transaccionales y <i>embeddings</i> en el mismo sistema.	El rendimiento en búsquedas vectoriales a gran escala (millones de vectores y alta concurrencia) suele ser inferior al de motores especializados.
	Aprovecha todo el ecosistema SQL: transacciones, <i>joins</i> , vistas, roles de usuario y mecanismos estándar de replicación y copia de seguridad.	Dispone de menos funcionalidades específicas de bases de datos vectoriales (gestión de colecciones, métricas especializadas o afinado fino de índices).
	Simplifica la operación en entornos reducidos o con restricciones de infraestructura, al requerir un único motor de base de datos.	La configuración óptima de índices y parámetros depende de la versión de <i>PostgreSQL</i> y de la extensión, y puede requerir un ajuste cuidadoso.

Tabla 4.2: Ventajas y desventajas de *Qdrant* y *pgvector*.

4.9. Desarrollo *web*

El desarrollo de la interfaz *web* para el proyecto se ha realizado con *Flask*⁵³ [7]. *Flask* es un *framework* WSGI (*Web Server Gateway Interface* [34] es un estándar *Python* para la comunicación entre los servidores *web* y el *framework web*) que proporciona los componentes esenciales para construir aplicaciones *web* usando *Python*. Fue diseñado para la creación de aplicaciones *web* ligeras y modulares.

Se ha optado por utilizar *Flask* para el desarrollo *web* en lugar de otros *frameworks*, como *Django*⁵⁴ o *FastAPI*⁵⁵ porque incorpora una serie de componentes que permiten gestionar las aplicaciones *web* de forma sencilla:

- Enrutamiento: permite asociar rutas URL a funciones *Python* utilizando decoradores. Lo cual permite organizar el código de manera modular. En el caso del proyecto simplifica la conexión entre la interfaz *web* y el *pipeline* de generación *RAG*.
- Gestión de peticiones y respuestas: proporciona objetos que permiten acceder de manera sencilla a los datos enviados por el usuario, utilizando formularios, JSON o simplemente parámetros.
- Motor de plantillas *Jinja2*: permite generar HTML a partir de datos procesados en el servidor. Lo cual facilita la visualización estructurada de las distintas partes del sistema.
- Soporte WSGI: como se ha comentado anteriormente, *Flask* se basa en este estándar para la comunicación entre aplicaciones *web* y servidores *Python*. La utilización de este estándar garantiza la compatibilidad con múltiples servidores de producción y facilita su despliegue mediante *Docker*.
- Extensiones: a pesar de que es minimalista, permite incorporar extensiones para bases de datos, autenticación o validación de formularios.

4.10. Evaluación de la calidad del código

Para evaluar y mantener la calidad del código desarrollado también se han incorporado herramientas de análisis estático. Concretamente se ha utilizado *SonarCloud*⁵⁶ y *Ruff*⁵⁷.

⁵³<https://flask.palletsprojects.com/en/stable/>

⁵⁴<https://www.djangoproject.com/>

⁵⁵<https://fastapi.tiangolo.com/>

⁵⁶<https://www.sonarsource.com/products/sonarqube/cloud/>

⁵⁷<https://docs.astral.sh/ruff/>

SonarCloud es una plataforma de análisis continuo integrada con GitHub que permite detectar automáticamente *bugs*, vulnerabilidades de seguridad, duplicidad de código, problemas de mantenibilidad, complejidad ciclomática y el *coverage* de las pruebas. Su utilización facilita identificar códigos que requieren refactorización y mejorar progresivamente la arquitectura de la aplicación.

Por su parte, Ruff es una herramienta ligera y muy rápida orientada al análisis estático y validación del estilo de código Python. Sirve para comprobar automáticamente errores de estilo, *imports* no utilizados, variables redundantes y malas prácticas de programación. Ayuda a mantener una estructura homogénea y coherente en todo el proyecto.

4.11. Despliegue de la aplicación

Para desplegar la *web* se han utilizado las herramientas *docker* (ver sección 4.9.1), *nginx* (ver sección 4.9.2), *gunicorn* (ver sección 4.9.3), DNS (ver sección 4.9.4) y el certificado SSL/TLS (ver sección 4.9.5). El uso de estas herramientas permite separar la responsabilidad y asegurar la portabilidad, el rendimiento y la seguridad.

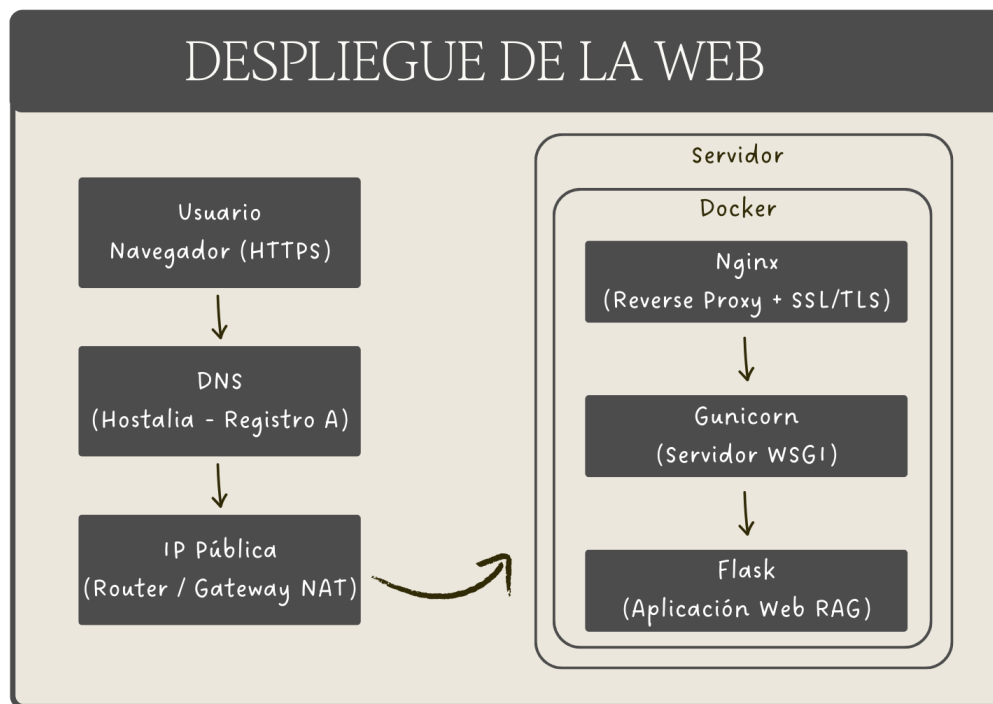
Empaquetar la aplicación en un contenedor *docker* garantiza su reproducibilidad, evitando conflictos entre librerías y versiones del sistema. *Gunicorn* sirve para no exponer directamente la aplicación *Flask* en Internet. *Nginx* se coloca por delante actuando como un servidor *web* y como *proxy* inverso. Va recibir las conexiones externas y las va a reenviar al servicio de *gunicorn*. También, se va a encargar de gestionar el cifrado y descifrado de HTTPS. El certificado SSL/TLS sirve para habilitar dicho protocolo HTTPS, en este caso demostrando control del dominio (DNS-01). En la imagen 4.2 se puede ver cómo se relacionan las distintas herramientas.

Docker

*Docker*⁵⁸ [9] es una plataforma de *software* de código abierto basada en contenedores. Permite empaquetar una aplicación con sus dependencias para distribuirla y ejecutarla, asegurando que funcione igual en cualquier entorno. Esto se debe a que aísla la aplicación y sus dependencias del sistema operativo dentro de unidades ligeras y portables (los contenedores). Los conceptos clave del *Docker* son:

⁵⁸<https://www.docker.com/>

Características	<i>PostgreSQL</i>	<i>SQLite</i>
Modelo	Cliente-servidor	Embebido
Concurrencia	Alta	Limitada (solo hay un escritor)
Escalabilidad	Alta	Baja
Uso recomendado	Sistemas multiusuario	Aplicaciones locales o prototipos
Integración con Docker	Muy buena	No necesaria (es un archivo local)

Tabla 4.3: Comparativa de *PostgreSQL* y *SQLite*.Figura 4.2: Relación entre las herramientas utilizadas para desplegar la aplicación *web*.

- Imagen: es una plantilla que contiene el sistema base y las dependencias necesarias.
- Contenedor: es una instancia en ejecución de la imagen.
- **Dockerfile** : es un archivo en el que se especifican los pasos que se deben seguir para construir la imagen.
- **Docker Compose** : es una herramienta que se define en un archivo `yml` en el cual se definen todos los servicios que debe contener la imagen. Estos servicios se dividen en volúmenes.

Se decidió implementar un contenedor *Docker* ya que mejora la reproducibilidad, la portabilidad y evita conflictos entre las dependencias en el sistema aportando mayor aislamiento. Además, permite integrar fácilmente diversas herramientas dentro de los volúmenes lo que mejora la escalabilidad. En sistemas que combinan múltiples componentes, como ocurre en este proyecto con LLM, base de datos vectorial y relacional e interfaz web, el uso de contenedores simplifica considerablemente la gestión de la infraestructura.

Nginx

*Nginx*⁵⁹ es un servidor *web* que puede actuar como *proxy* inverso [3]. Un *proxy* inverso es un servidor que, a diferencia de un *proxy* normal, se sitúa delante de los servidores *web* para interpretar las solicitudes de los clientes. Se encarga de reenviar las solicitudes de los clientes a los servicios *web*. En el caso del proyecto *Nginx* se ha utilizado como intermediario entre el cliente y la aplicación *Flask* para recibir las peticiones externas (HTTP/ HTTPS). También, se encarga de gestionar el certificado SSL y de redirigir el tráfico hacia *Gunicorn*.

La ventaja de utilizar *Nginx* es que está optimizado para manejar múltiples conexiones concurrentes de forma eficiente. Además, desacopla el servidor *web* del servidor de aplicaciones, aumentando la seguridad y el rendimiento del sistema.

Gunicorn

*Gunicorn*⁶⁰ es un servidor WSGI para aplicaciones *Python* fácil de instalar (no requiere dependencias adicionales), pero que no es compatible con *Windows*. Su principal función es ejecutar la aplicación *Flask* en un entorno preparado para producción. Ya que, por defecto, el *framework* *Flask*

⁵⁹<https://nginx.org/>

⁶⁰<https://gunicorn.org/>

incluye un servidor de desarrollo que no es adecuado para entornos reales. Por lo cual *Gunicorn* se encarga de sustituir este servidor por uno optimizado para producción que si que va a permitir gestionar múltiples procesos de manera simultánea y que va a mejorar la estabilidad.

DNS

El DNS (*Domain Name System*) permite asociar un nombre de dominio a la dirección IP del servidor donde se encuentra desplegada la aplicación. Asignarle un dominio a una aplicación sirve para facilitar el acceso y otorgarle mayor profesionalidad. Además, de mejorar la accesibilidad, permite la migración del servidor sin necesidad de cambiar la dirección que utiliza el usuario.

En este proyecto el dominio público se ha adquirido a través del proveedor *Hostalia*⁶¹. Una vez registrado el dominio, fue necesario configurar sus registros DNS para asociarlo con la dirección IP del servidor donde se encuentra desplegada la aplicación. Para vincular el dominio al servidor se configuró un registro tipo A dentro del panel de gestión DNS de *Hostalia*. Este registro permite que las peticiones dirigidas al dominio se redirijan a la dirección IP correspondiente del servidor.

Sin embargo, la aplicación se ejecuta en una IP privada de la Universidad de Burgos. Las direcciones IP privadas no son accesibles directamente desde Internet [24], ya que están destinadas a redes internas. Para conseguir que la aplicación sea accesible desde Internet hay que configurar el *router* de tal forma que el dominio apunte a la IP pública del dicho *router*. Mediante técnicas NAT (*Network Address Translation* [5]) se redirige el tráfico hacia la IP privada del servidor.

Certificado SSL/TLS

Para garantizar la seguridad de las comunicaciones se ha configurado un certificado SSL/TLS que permite el uso del protocolo HTTPS (*HyperText Transfer Protocol Secure*) [2]. HTTPS es la versión segura de HTTP y se basa en el protocolo TLS (*Transport Layer Security*), cuyo objetivo es cifrar la comunicación entre el cliente y el servidor. Este cifrado impide que terceros puedan interceptar, modificar o suplantar la información transmitida, protegiendo así la confidencialidad e integridad de los datos.

⁶¹<https://www.hostalia.com/>

En este proyecto, el certificado digital se ha emitido utilizando *Certbot*⁶² que es una herramienta oficial promovida por la *Electronic Frontier Foundation*⁶³ (EFF). *Certbot* automatiza la obtención e instalación de certificados gratuitos proporcionados por *Let's Encrypt*⁶⁴. La cual es una autoridad certificadora reconocida internacionalmente. El proceso de emisión del certificado se basa en un mecanismo de validación del dominio, mediante el cual se demuestra que el servidor tiene control sobre el dominio solicitado. Una vez validado, se genera un certificado digital que incluye la clave pública del servidor, la identidad del dominio y la firma digital de la autoridad certificadora. Concretamente se ha utilizado el método de validación DNS-01 que consiste en demostrar que se tiene control sobre el dominio añadiendo un registro específico tipo `TXT` en la configuración DNS. Este registro contiene un *token* generado por la autoridad certificadora.

⁶²<https://certbot.eff.org/>

⁶³<https://www.eff.org/>

⁶⁴<https://letsencrypt.org/>

5. Aspectos relevantes del desarrollo del proyecto

En este apartado se van a explicar los aspectos más importantes del proyecto. Se van a recoger los distintos retos se han encontrado durante el desarrollo indicando como se solventaron. También, se van a explicar las decisiones más relevantes que se han tomado para llevar a cabo el proyecto.

5.1. Inicio del proyecto

Actualmente la mayor parte de las personas utilizan procesadores del lenguaje, ya bien sea a través de [ChatGPT](https://chat.openai.com)⁶⁵, [Claude](https://claude.ai)⁶⁶, [Gemini](https://gemini.google.com/app)⁶⁷ o [NotebookLM](https://notebooklm.google.com/)⁶⁸, pero la mayor parte de los usuarios no conocen lo suficiente de su funcionamiento como para sacarle el máximo partido. Generalmente los *prompts* de estos usuarios son escuetos, sin el contexto adecuado, lo que permite que el modelo alucine en sus respuestas dándoles información errónea que generalmente no contrastan. En este contexto surgió la idea del proyecto, generar un *LLM* especializado en licitaciones del estado, concretamente en las de la Junta de Gobierno de la Diputación Provincial de Burgos.

Para evitar que fuese el usuario el que le tuviera que otorgar el contexto al modelo se decidió implementar un *RAG* que contuviera todas las licitaciones de la Junta de Gobierno de la Diputación Provincial de Burgos publicadas en la página de [Contratación del Sector Público](https://tinyurl.com/48ay5679)⁶⁹. Además, para evitar

⁶⁵<https://chat.openai.com>

⁶⁶<https://claude.ai>

⁶⁷<https://gemini.google.com/app>

⁶⁸<https://notebooklm.google.com/>

⁶⁹<https://tinyurl.com/48ay5679>

alucinaciones del modelo se decidió limitar sus respuestas a la información que le proporciona el *RAG*.

Otro problema típico de los *LLM* es la fecha de entrenamiento, ya que no son capaces de contestar preguntas posteriores a esta fecha, ya que no conocen su respuesta. Para evitar este problema de desactualización se decidió incluir un programa de *web scraping* para que se pueda extraer de forma periódica las nuevas licitaciones y volver a descargar aquellas que hayan sido actualizadas.

5.2. Web Scraping

Tras formalizar la idea se comenzó con el proyecto realizando la aplicación de *Web Scraping* para extraer las licitaciones de la página de **Contratación del Sector Público**⁷⁰ de manera automática. Se desarrollaron diferentes soluciones utilizando *Selenium* y *Playwright*, optando finalmente por el uso de *Playwright* en modo asíncrono, ya que permite gestionar de forma automática las esperas y la interacción con elementos dinámicos, haciendo que el sistema sea más robusto frente a fallos y *timeouts* que con los programas realizados con *Selenium*. Además, al usar *Playwright* asíncrono (*asyncio*) se pudo incorporar procesamiento concurrente, ya que permite procesar varias tareas en paralelo, para optimizar el tiempo de extracción y descarga de datos.

La elevada complejidad de la página debido al uso de contenido dinámico e *iframes* supuso un reto. Para solventarlo se tuvieron que desarrollar mecanismos específicos para la localización de elementos y la navegación entre distintos contextos. Finalmente, los datos obtenidos se normalizan y almacenan en un archivo `JSON`. Este archivo se usa para descargar los pliegos finales.

Cabe destacar que durante el desarrollo del proyecto se actualizó la página **Contratación del Sector Público**⁷¹, por lo cual hubo que realizar cambios en el programa ya desarrollado, modificando selectores y flujos de navegación para mantener su funcionamiento.

⁷⁰<https://tinyurl.com/48ay5679>

⁷¹<https://tinyurl.com/48ay5679>

5.3. *Retrieval Augmented Generation (RAG)*

Durante el desarrollo del sistema *RAG* se han evaluado distintos enfoques para cada uno de sus componentes principales, con el objetivo de optimizar la calidad de las respuestas, la eficiencia del sistema y su facilidad de despliegue. A continuación, se describen los elementos más relevantes analizados, las pruebas realizadas y la justificación de las decisiones finales adoptadas.

Tokenizer

Uno de los primeros aspectos analizados fue el uso del *tokenizer*, ya que es el encargado de transformar el texto en unidades básicas (*tokens*) que son la entrada real de los modelos de lenguaje (*LLM*). Su correcto funcionamiento es clave tanto para el procesamiento del texto como para el control del tamaño de los fragmentos (*chunks*).

Se realizaron pruebas con diversos *tokenizers* asociados a distintos modelos de lenguaje (como *LLaMA* o *Mistral*), así como con el *tokenizer* incluido en los modelos de *embeddings*. Se observó que utilizar el *tokenizer* del propio modelo de *embeddings* simplifica el *pipeline* y evita inconsistencias entre la representación del texto y su vectorización semántica. Además, permite conocer directamente el número máximo de *tokens* admitido por el modelo, facilitando el control del tamaño de entrada y evitando superar la ventana máxima soportada.

Por este motivo, se optó por utilizar el *tokenizer* del modelo de *embeddings* (`SentenceTransformer`), garantizando coherencia entre el proceso de *chunking* y la generación de *embeddings*.

En la implementación final, el sistema obtiene automáticamente tanto el *tokenizer* como el tamaño máximo de secuencia directamente desde el modelo cargado, permitiendo adaptar dinámicamente el sistema si se cambia de modelo de *embeddings*. Esto facilita la reutilización del *pipeline* con distintos modelos sin necesidad de modificar manualmente los parámetros de tokenización.

Chunking

El *Chunking* consiste en dividir los documentos en fragmentos manejables. Es fundamental para ajustarse a la ventana de contexto del modelo y para

mejorar la recuperación semántica. El texto se puede segmentar por longitud fija (*tokens*), por secciones o párrafos o por significado semántico.

Este proceso fue uno de los procesos más experimentados, probándose diferentes estrategias como el *Chunking* por tamaño fijo de caracteres, por *tokens* con *overlap* (solapamiento) para mantener el contexto o basado en secciones o frases. Durante las pruebas se comprobó que el uso exclusivo de *chunks* por tamaño fijo generaba fragmentos poco naturales, afectando negativamente a la recuperación. Por otro lado, el *chunking* semántico puro resultaba más complejo y costoso.

Por ello se optó por una estrategia híbrida, en la cual primero se realiza una agrupación lógica del texto utilizando frases y separadores naturales del documento y posteriormente se divide en *chunks* controlados por número de *tokens*, incorporando solapamiento entre fragmentos consecutivos. Este solapamiento permite conservar información contextual relevante entre fragmentos cercanos, reduciendo la pérdida de información cuando una explicación queda dividida entre dos *chunks*.

Además, durante el desarrollo se realizaron pruebas variando tanto el tamaño de los *chunks* como el porcentaje de solapamiento. Se observó que *chunks* demasiado pequeños reducían la capacidad contextual del modelo, mientras que fragmentos demasiado grandes empeoraban la precisión de la recuperación y aumentaban el número de *tokens* enviados al *LLM*. La estrategia híbrida seleccionada consiguió un equilibrio entre coherencia semántica, precisión en la recuperación y eficiencia computacional.

Embeddings

Los *embeddings* son representaciones vectoriales del texto que capturan su significado semántico. Permiten comparar las consultas de los usuarios con los fragmentos de los documentos y realizar búsquedas por similitud. Son la base del funcionamiento del sistema de recuperación del *RAG*. Por lo cual para elegir el modelo final se probaron varios comparando su calidad semántica, la velocidad de ejecución y el consumo de recursos. Finalmente, se seleccionó el modelo `sentence-transformers/all-MiniLM-L6-v2` debido a que ofrece buen equilibrio entre rendimiento y coste computacional. Además, su integración mediante `SentenceTransformer` es sencilla y su dimensión de *embedding* es suficiente para tareas de recuperación semántica sobre documentos administrativos.

Durante las pruebas también se evaluó el comportamiento del sistema en CPU y GPU. La implementación final permite detectar automáticamente el

dispositivo disponible y ejecutar el cálculo de *embeddings* utilizando aceleración por GPU cuando está disponible. Además, se incorporaron mecanismos de recuperación ante errores de memoria en CUDA, permitiendo continuar automáticamente la generación de *embeddings* en CPU si la memoria de la GPU resulta insuficiente.

Otra mejora relevante implementada fue el procesamiento por lotes (*batch processing*) durante la generación de *embeddings*. En lugar de vectorizar cada *chunk* individualmente, el sistema procesa múltiples fragmentos simultáneamente, reduciendo significativamente el tiempo de indexación.

Base de datos vectorial

La base de datos vectorial es la encargada de almacenar los *embeddings* y permitir realizar búsquedas en espacios de alta dimensión. Tras estudiar diversas opciones (ver sección 4.8.1) se eligió *Qdrant* ya que, al tener soporte nativo para la búsqueda vectorial simplifica mucho su uso eficiente. Además, permite almacenar metadatos junto con los *chunks* y *embeddings*, lo que se consideró de utilidad para aumentar el contexto durante la recuperación. Como metadatos se incluyen los *hashes* SHA-256 de los documentos originales para detectar automáticamente cuándo un documento ha cambiado y reindexarlo sin duplicar información.

Durante el desarrollo se implementó un sistema de creación automática de colecciones en *Qdrant*. Estas colecciones están configuradas con búsqueda por similitud coseno y una dimensión de vector ajustada dinámicamente al modelo de *embeddings* empleado. Para optimizar el rendimiento, los *chunks* se almacenan mediante operaciones *bulk upsert*, reduciendo significativamente el tiempo de indexación frente a inserciones individuales. Además, se utiliza una capa de abstracción (similar a un *Object-Vector Mapping*) para encapsular la lógica de conexión, serialización y recuperación de datos desde *Qdrant*. Esta capa permite desacoplar parcialmente el sistema del motor vectorial.

Retriever (mecanismo de recuperación)

Es el componente encargado de buscar en la base vectorial los fragmentos más relevantes para responder a una consulta. Su calidad es crítica, ya que determina el contexto que se proporcionará al modelo. Por lo tanto, la calidad de la respuesta depende directamente de la capacidad del *retriever* para recuperar información relacionada con la pregunta del usuario. Se basa en la búsqueda por similitud vectorial, principalmente por similitud coseno.

En los primeros prototipos del proyecto, el sistema solo recuperaba el *chunk* más parecido a la consulta del usuario. Para ello, primero se convertía la pregunta en un *embedding* (utilizando el mismo modelo que el que se usa para indexar los documentos). Posteriormente, se calculaba la similitud entre dicho *embedding* y los almacenados en la base de datos vectorial. Finalmente, se seleccionaba el fragmento con mayor similitud y este se incorporaba directamente al *prompt* enviado al *LLM*. Este enfoque se planteó como punto de partida para validar el funcionamiento básico del *RAG* pero presentaba limitaciones importantes en la generación de la respuesta, haciendo que la mayoría de las preguntas no las pudiese contestar.

Después, se probó a cambiar el número de *chunks* recuperados (**top-k retrieval**). Se observó que recuperar pocos *chunks* produce respuestas demasiado limitadas, llevando a generalizaciones que no se cumplen en todos los documentos. En cambio, recuperar demasiados introduce ruido al modelo haciendo que responda con información irrelevante. Además, se vio que los contextos demasiado grandes hacen que el modelo utilice más *tokens* para contestar, empeorando la latencia de generación.

Otro aspecto relevante fue al evolución del propio *chunking*. Al principio los documentos se dividían utilizando tamaños fijos de texto. Sin embargo, se comprobó que esta estrategia fragmentaba información relacionada y afectaba negativamente a la recuperación semántica. Posteriormente, se introdujo el uso de solapamiento entre los *chunks*, permitiendo compartir parte del contenido entre fragmentos consecutivos. Esta mejora incrementó significativamente la coherencia del contexto recuperado y redujo la pérdida de información relevante entre fragmentos contiguos.

Además, del contenido textual, también se añadieron metadatos asociados a cada *chunk* como el nombre del documento, el título de la licitación, el identificados del fragmento, la posición dentro del documento y la información estructural obtenida durante el preprocesado. Utilizar metadatos mejoró la trazabilidad de las respuestas generadas.

Finalmente el sistema utiliza una estrategia de recuperación dinámica basada en búsqueda vectorial por similitud coseno sobre Qdrant. Para cada consulta, la pregunta del usuario se transforma en un *embedding* y posteriormente se calcula la similitud con todos los *embedding* almacenados en la base vectorial. Pero, ahora el sistema recupera un conjunto de *chunks* los cuales deben tener una similitud con la pregunta de al menos 0'5 para descartar fragmentos poco relacionados con la consulta. El número de *chunks* recuperados depende del tipo de pregunta realizada, las consultas generales utilizan hasta 20 fragmentos, mientras que las consultas guiadas tienen con-

figuraciones específicas optimizadas para cada caso. Por ejemplo, preguntas orientadas a resúmenes globales pueden recuperar hasta 80 chunks para proporcionar una visión completa del documento, mientras que consultas más concretas, como solvencia, garantías o duración contractual, utilizan entre 12 y 18 fragmentos para reducir el ruido y mantener la precisión. Además, de la recuperación semántica se aplican filtros por metadatos, como número de expediente o tipo de documento (administrativo/técnico) para limitar la búsqueda únicamente a los documentos relevantes. Este enfoque híbrido permite equilibrar cobertura contextual, precisión semántica y eficiencia en el consumo de tokens del LLM.

Augmentation

Este proceso consiste en combinar la consulta del usuario con los fragmentos recuperados para construir un *prompt* enriquecido. Esta fase incluye técnicas de ingeniería de *prompts* para mejorar la calidad de la respuesta. En el desarrollo del *RAG* se probaron distintas estrategias, observándose diferencias significativas en el comportamiento del sistema.

En las estrategias más simples, los *chunks* recuperados se concatenaban junto con la pregunta del usuario. Aunque este enfoque era sencillo, el modelo tendía a mezclar información o a ignorar parte del contexto recuperado.

Observadas estas limitaciones se decidió utilizar *prompts* estructurados. En los cuales se separa el contexto recuperado, la pregunta del usuario, las instrucciones del sistema y las restricciones sobre como responder.

Con este enfoque el *LLM* interpreta mejor el contexto recuperado y disminuye las alucinaciones. Además, se añadieron instrucciones específicas indicando que la respuesta deben basarse únicamente en la información proporcionada por los *chunks* recuperados.

También, se han realizado pruebas modificando el orden de los fragmentos recuperados dentro del *prompt*. Se observó que colocar primero los *chunks* con mayor similitud mejora la precisión de las respuestas, especialmente en consultas complejas o ambiguas.

Otra mejora importante fue la incorporación de referencias documentales dentro del *prompt*. En lugar de proporcionar únicamente texto plano, cada fragmento incluye información sobre el documento de origen. Esto ayuda al modelo a contextualizar mejor la información y facilita mostrar al usuario la procedencia de la respuesta generada.

Finalmente, el sistema incorpora distintos *prompts* para contestar dependiendo del tipo de consulta que haga el usuario. Se ha diseñado un *prompt* general utilizado para preguntas abiertas, pero, además, se han definido plantillas específicas para tareas concretas como resúmenes globales del documento, extracción de importes económicos, plazos, solvencia, criterios de adjudicación, garantías, presupuestos, duración contractual, penalizaciones o presentación de ofertas. Cada plantilla incluye instrucciones adaptadas al tipo de información buscada y define una estructura concreta para la respuesta esperada. Por ejemplo, los *prompts* de tipo resumen solicitan una explicación global y estructurada del pliego, mientras que los orientados a criterios de adjudicación fuerzan al modelo a identificar ponderaciones, fórmulas y subcriterios. Además, el sistema utiliza un *system prompt* por defecto que obliga al modelo a responder en español de forma breve y precisa. Esta combinación de *prompts* especializados y selección dinámica del contexto permite adaptar el comportamiento del *LLM* al tipo de consulta realizada, mejorando la precisión, consistencia y utilidad de las respuestas generadas.

LLM

Es el encargado de generar la respuesta final utilizando el contexto proporcionado por el sistema *RAG*. La elección del modelo (tamaño, arquitectura, si es local o una *API*) afecta directamente a la calidad, latencia y coste del sistema.

Durante el desarrollo del proyecto, la integración de los modelos de lenguaje fue evolucionando progresivamente. Se comenzó realizando pruebas utilizando modelos locales (obtenidos de Hugging Face) ejecutados mediante Ollama. Estas pruebas permitieron comprobar la viabilidad de ejecutar modelos localmente sin depender de ninguna *API* externa, lo que aporta un mayor control sobre el sistema y evita costes asociados al uso de servicios comerciales.

Inicialmente se realizaron pruebas con diversos modelos locales como *Mistral* o *LLaMA* para probar distintas arquitecturas y tamaños. Gracias a estas pruebas se pudo ver como afecta el tamaño a la respuesta generada y al tiempo invertido. Se vio que los modelos más pequeños tardaban menos en generar las respuestas, pero dichas respuestas eran de menor calidad. Tendían a ser breves y menos precisas. Mientras que los modelos más grandes generaban mejores respuestas, pero con mayor consumo de recursos y latencias. Finalmente, se seleccionó como modelo principal del sistema *LLaMA* 3.1-8B, ya que tiene buen rendimiento en tareas de comprensión y

generación. Este modelo ofrece un buen equilibrio entre la claridad de la respuesta generada, la utilización del contexto recuperado y los requisitos de *hardware*. Otro aspecto que se tuvo en cuenta a la hora de elegir el modelo fue que permite la ejecución en local usando Ollama. Lo que permite mantener el control total sobre el sistema sin dependencias externas.

Finalmente, se decidió incluir soporte para múltiples modelos configurables desde la propia aplicación *web*. De tal forma que la arquitectura final permite seleccionar dinámicamente entre tres modelos distintos (*LLaMA* 3.1-8B, *gemma-4B* y *qwen3-4B*, cargados desde Ollama) integrados dentro del sistema RAG. Esta evolución surgió debido a la necesidad de comparar comportamientos y ofrecer mayor flexibilidad dependiendo del tipo de consulta realizada.

Para soportar esta funcionalidad fue necesario desacoplar la lógica de generación del resto del *pipeline RAG*, implementando una capa de abstracción que permitiese cargar dinámicamente los distintos modelos y modificar sus parámetros. Además, se añadieron funciones auxiliares para gestionar las configuraciones disponibles y mostrar automáticamente las opciones en la interfaz web.

Además, se han ajustado parámetros de generación como el número máximo de *tokens* generados, la longitud máxima de entrada, la temperatura, el formato del *prompt* y el número de *chunks* recuperados. Lo que permitió mejorar la calidad de las respuestas, reduciendo alucinaciones y adaptandolas al contexto del sistema.

5.4. Conversión a *Markdown*

La conversión de los documentos PDF a Markdown permite transformar documentos administrativos complejos en texto estructurado apto para el proceso de *chunking*, generación de *embeddings* y recuperación semántica. Inicialmente, los documentos PDF se procesaban extrayendo texto plano. Este enfoque presentaba limitaciones como la pérdida de la estructura del documento, lo que provocaba que el sistema no pudiese diferenciar los títulos, subtítulos, tablas o las secciones. Por este motivo, durante el desarrollo se comenzó a investigar la posibilidad de convertir los PDF a Markdown de forma automática.

Las primeras aproximaciones consistieron en utilizar bibliotecas tradicionales de extracción de texto en Python. Estas herramientas permitían obtener el contenido textual de los PDF, pero perdían gran parte del formato

original. Uno de los principales problemas encontrados fue que los documentos de licitaciones públicas presentan formatos extremadamente heterogéneos. Debido a esto se comenzaron a realizar pruebas utilizando OCR (*Optical Character Recognition*). Sin embargo, aunque el OCR permitía recuperar texto que anteriormente no podía extraerse, seguía presentando problemas. No se reconocían correctamente todos los caracteres, se seguía perdiendo parte del formato, las tablas no siempre se construían bien. Además, el uso de OCR supuso un incremento considerable del tiempo de procesamiento.

La implementación final se basa en un sistema de OCR multimodal que utiliza modelos visuales ejecutados mediante *Ollama*, concretamente el modelo `blaifa/Nanonets-OCR-s`. Este modelo recibe imágenes de las páginas del PDF junto con un *prompt* especializado que fuerza la generación de Markdown estructurado, incluyendo títulos jerárquicos, tablas HTML, ecuaciones en $\text{\LaTeX}$, listas y otros elementos relevantes del documento original.

El procesamiento se realiza de forma asíncrona página a página. Primero, cada página del PDF se convierte individualmente en una imagen utilizando `pdf2image`, evitando cargar el documento completo en memoria y reduciendo así el consumo de recursos. Posteriormente, las páginas se procesan concurrentemente mediante tareas asíncronas controladas con semáforos, permitiendo paralelizar parcialmente el OCR y mejorar el rendimiento global del sistema. Esta arquitectura resulta especialmente importante debido al elevado tiempo de procesamiento de los documentos administrativos extensos.

Además, la implementación está preparada para utilizar aceleración por GPU cuando está disponible. El sistema detecta automáticamente la configuración de GPU y solicita a *Ollama* la ejecución del modelo visual utilizando todas las capas posibles en GPU. También se incorporaron mecanismos de *fallback* a CPU en caso de error, así como reintentos automáticos reduciendo progresivamente la resolución de las imágenes cuando el modelo produce fallos de memoria o errores internos. Estas optimizaciones fueron necesarias debido al elevado consumo computacional del OCR multimodal, especialmente en documentos con muchas páginas o tablas complejas.

Tras la conversión inicial, el Markdown generado pasa por una fase adicional de normalización automática. En esta etapa se corrigen índices con puntos suspensivos, se detectan encabezados numerados y se transforman en títulos Markdown jerárquicos, incluso cuando el modelo OCR no los reconoce correctamente. Para ello se desarrollaron reglas específicas adaptadas a la estructura habitual de los pliegos.

Durante el desarrollo se observó que esta aproximación mejora considerablemente la calidad del texto utilizado posteriormente por el sistema *RAG*, especialmente frente a OCR tradicionales basados únicamente en extracción plana de texto. Sin embargo, también se detectaron varias limitaciones importantes. La principal es el elevado tiempo de procesamiento, ya que cada página debe renderizarse como imagen y posteriormente analizarse mediante un modelo multimodal. Además, aunque el sistema consigue detectar correctamente gran parte de la estructura documental, no siempre identifica de forma precisa todas las secciones, tablas o jerarquías debido a que los formatos de los documentos son poco homogéneos.

5.5. Evaluación *RAG*

Un aspecto relevante del proyecto fue incluir una evaluación del sistema *RAG* implementado. Para dicha evaluación se ha utilizado un *pipeline* de evaluación *RAG* propio. El cual se inspira en *frameworks* existentes como ARES (*Automated RAG Evaluation System*) y RAGAS (*Retrieval-Augmented Generation Assessment*). El objetivo principal era construir un sistema de evaluación reproducible, automatizable y completamente local, evitando dependencias de API externas y garantizando la confidencialidad de los documentos utilizados.

La arquitectura implementada combina distintos enfoques de evaluación con la generación automática de *datasets* de evaluación. Para la generación de dichos *datasets* se utiliza ARES, el cual genera de manera automática preguntas, respuestas y evidencias documentales a partir de los propios *chunks* almacenados en la base de datos vectorial.

Respecto a la evaluación, por un lado, se utiliza RAGAS para calcular métricas semánticas relacionadas con la calidad de la recuperación y de la generación utilizando un enfoque *LLM-as-a-judge*. Las métricas utilizadas son *Faithfulness*, *Answer relevance*, *Answer correctness*, *Context precision y recall* y *Context relevance*. Las cuales permiten analizar si la respuesta realmente está fundamentada en el contexto recuperado y si dicho contexto es adecuado para responder la consulta. Esta evaluación se combina con métricas de respaldo basadas en *embeddings* y similitud coseno. Estas métricas permiten calcular automáticamente el grado de similitud semántica entre preguntas, respuestas, contexto recuperado y *ground truth* utilizando representaciones vectoriales. Finalmente, ambas métricas se combinan mediante medias ponderadas. Consiguiendo una evaluación híbrida más robusta y estable. Esta combinación permitió reducir la variabilidad típica de los siste-

mas basados únicamente en *LLM-as-a-judge* y mejorar la reproducibilidad de los resultados.

5.6. Desarrollo *web*

Tras realizar un primer prototipo *RAG* se comenzó el diseño de una aplicación *web* para permitir a los usuarios interactuar con el sistema de manera sencilla y visual. Para ello se decidió utilizar Flask como *framework* de desarrollo *web*.

Flask es un *framework* ligero para Python basado en la arquitectura WSGI que permite desarrollar aplicaciones web de manera modular y escalable. Una de las razones principales para su elección fue su flexibilidad, ya que facilita integrar distintos componentes del sistema *RAG*, como el *retriever*, el procesamiento documental o los modelos *LLM*, dentro de una misma aplicación. Además, Flask permite estructurar el proyecto utilizando *blueprints*, lo que facilitó dividir la aplicación en módulos independientes y mejorar su mantenibilidad.

Durante las primeras fases del desarrollo *web* se realizaron prototipos muy simples cuyo objetivo principal era únicamente permitir realizar consultas al sistema *RAG* desde una interfaz básica. Estas primeras versiones consistían en páginas HTML estáticas con formularios mínimos para introducir preguntas y mostrar respuestas del modelo. Conforme el proyecto evolucionó, la aplicación fue creciendo progresivamente hasta convertirse en una plataforma web completa con autenticación, gestión documental, historial de consultas, paneles administrativos y procesamiento asíncrono de tareas.

Desde el punto de vista arquitectónico, la aplicación se divide en *frontend* y *backend*. El *frontend* se encarga de la interacción visual con el usuario y el *backend* se encarga de la lógica de negocio, procesamiento documental, acceso a bases de datos y comunicación con el sistema *RAG*.

Frontend

El *frontend* de PythIA ha evolucionado respecto a los primeros prototipos desarrollados durante el proyecto. Las primeras interfaces eran simples y estaban centradas en validar el funcionamiento técnico del sistema *RAG*. Sin embargo, conforme el proyecto avanzó, se comenzó a trabajar en el diseño de una interfaz más moderna, usable y adaptada a distintos tipos de usuarios. En la cual la complejidad del sistema *RAG* es transparente para el usuario.

El diseño visual de la aplicación se ha ido desarrollando a lo largo de todo el proyecto mediante distintas iteraciones de las páginas. Uno de los cambios más importantes respecto a los primeros prototipos fue la incorporación de soporte completo para modo claro y modo oscuro (dejando por defecto el modo del sistema salvo que el usuario indique otra cosa en su perfil) mejorando así la accesibilidad y la experiencia de usuario.

Para implementar la interfaz se utilizó Bootstrap como *framework* CSS principal. Bootstrap permitió acelerar el desarrollo del *frontend* gracias a su sistema de componentes reutilizables, facilitando mantener una coherencia visual entre todas las páginas de la aplicación. La utilización de Bootstrap también permitió implementar un diseño *responsive* para que los usuarios puedan usar la aplicación desde distintos dispositivos.

Otro aspecto importante del *frontend* fue la internacionalización de la aplicación. Para esto se ha desarrollado un sistema de traducción basado en diccionarios de claves y funciones auxiliares que permiten traducir los textos de la interfaz, los formularios, los mensajes de error, las validaciones, las notificaciones y los componentes dinámicos del JavaScript. Esto permitió desacoplar completamente los textos de la interfaz del código HTML y Python.

Además del diseño visual, también se incorporaron componentes gráficos interactivos utilizando D3.js. Lo que permitió generar visualizaciones mucho más dinámicas y personalizables. Además, facilitó adaptar las gráficas al diseño visual de la aplicación y a los distintos temas claro y oscuro.

Por otro lado, conforme el sistema fue incorporando más funcionalidades, se observó que algunos usuarios podían tener dificultades para comprender ciertos flujos de uso. Para mejorar la experiencia de usuario se han incorporado Driver.js como sistema de tutoriales interactivos guiados. Facilitando el aprendizaje inicial de la plataforma y mejorando la accesibilidad del sistema.

Backend

El *backend* de PythIA es el núcleo funcional de la aplicación y el encargado de integrar todas las partes del sistema *RAG*. Durante el desarrollo ha ido evolucionando desde *scripts* independientes hasta una arquitectura modular basada en Flask y organizada mediante *blueprints*, servicios y capas desacopladas.

Sus funcionalidades son:

- Gestión de usuarios.

- Registro e inicio de sesión.
- Recuperación de contraseñas.
- Gestión documental.
- Importación automática de licitaciones mediante *web scraping*
- Conversión de documentos.
- Generación de *embeddings*
- Indexación vectorial.
- Consultas del sistema *RAG*.
- Historial de consultas
- Estadísticas de uso y de comparación de modelos

La aplicación se organizó utilizando controladores Flask agrupados mediante *blueprints*. Cada *blueprint* encapsula un conjunto concreto de funcionalidades, como autenticación, administración, consultas *RAG*, gestión documental y páginas principales. Esta organización ha permitido desacoplar las distintas partes del sistema y mejorar la escalabilidad del proyecto.

Por debajo de los controladores se implementó una capa de servicios encargada de la lógica de negocio. Esta separación fue especialmente importante conforme el proyecto fue creciendo, ya que evita sobrecargar los controladores Flask con lógica de negocio. Permitiendo reutilizar código entre distintas partes de la aplicación y facilitando el mantenimiento. Concretamente se han implementado los siguientes servicios:

- Comunicación con los *LLM*.
- Recuperación semántica.
- Generación de *embeddings*.
- Conversión a Markdown.
- Procesamiento de PDF.
- *Web scraping*.
- Gestión de tareas asíncronas.
- Gestión de documentos.
- Procesamiento del historial.

En cuanto a los modelos de datos, la aplicación utiliza SQLAlchemy como ORM principal. Los modelos representan entidades como:

- Usuarios
- Consultas
- Documentos
- Chunks

- **Embeddings**
- Estados de tareas
- Configuraciones del sistema

Además de la base de datos relacional utilizada para la gestión general de la aplicación, el *backend* también integra Qdrant como base de datos vectorial para almacenar los *embeddings* utilizados por el sistema *RAG*.

La comunicación entre las distintas capas de la aplicación sigue una arquitectura multicapa donde el usuario interactúa con el *frontend*. A continuación los controladores reciben las peticiones y los servicios ejecutan la lógica de negocio. Los modelos gestionan el acceso a los datos y los resultados se devuelven al *frontend*.

Otro aspecto importante fue la seguridad y el control de acceso. La aplicación implementa autenticación mediante Flask-Login y utiliza distintos roles de usuario para restringir determinadas funcionalidades administrativas. Entre las medidas de seguridad implementadas destacan:

- *Hash* seguro de contraseñas.
- Protección CSRF en formularios.
- Variables de entorno para claves privadas.
- Restricción de acceso por roles.
- Validación de formularios.
- Gestión segura de sesiones.
- Protección de rutas mediante decoradores personalizados.

También se ha desarrollado un sistema centralizado de control de excepciones para evitar errores no controlados dentro de la aplicación. Esto permitió:

- Mostrar mensajes de error amigables.
- Registrar errores mediante *loggers*.
- Facilitar la depuración.
- Evitar caídas completas del sistema.

Finalmente, debido al elevado coste computacional de algunas operaciones, se incorporó soporte para tareas asíncronas. Esto fue especialmente importante en procesos como la conversión de PDF a Markdown, el *web scraping*, la generación de *embeddings*, la indexación documental y la generación de la respuesta. Permite evitar bloqueos en la interfaz web, ya que los

procesos pesados pueden ejecutarse en segundo plano mientras el usuario continua usando la aplicación.

5.7. Evaluación de la calidad del código

A medida que el proyecto fue creciendo y la aplicación pasó de estar formada por pequeños prototipos experimentales a convertirse en un sistema *web* completo surgió la necesidad de incorporar mecanismos que permitiesen evaluar y controlar la calidad de código desarrollado.

Durante las primeras fases del proyecto, la prioridad principal era desarrollar *scripts* funcionales sin tener en cuenta el código duplicado, las funciones demasiado complejas, la escasa modularización, la baja reutilización o las dependencias acopladas. Para facilitar la mantabilidad y la escalabilidad se decidió incorporar herramientas automáticas de análisis estático de código.

Concretamente se decidió usar **SonarCloud**⁷². La cual se integró directamente con el repositorio GitHub del proyecto, para analizar automáticamente distintos indicadores relacionados con la calidad del software. Su uso permitió detectar y solucionar *bugs*, vulnerabilidades de seguridad, código duplicado, funciones extremadamente complejas, funciones demasiado largas, variables sin utilizar, problemas de mantenibilidad, incumplimientos de estándares de calidad y problemas con tipado y control de excepciones.

Como consecuencia de estos análisis, se realizaron diversas refactorizaciones importantes dentro del proyecto. Por ejemplo se realizó una separación más estricta entre los controladores y los servicios, se modularizaron las funciones más complejas, se reutilizó la lógica común reduciendo el código duplicado y se mejoró el control de excepciones.

Además, de SonarCloud también se utilizó **Ruff**⁷³ como herramienta complementaria para el análisis estático y mejora de la calidad del código. Ruff se integró dentro del flujo de desarrollo para realizar comprobaciones rápidas relacionadas con el estilo, complejidad y consistencia del código. Su principal ventaja es su gran velocidad de ejecución que permitió utilizarlo frecuentemente durante el desarrollo sin afectar significativamente al tiempo de trabajo. Gracias a Ruff se detectaron automáticamente problemas como *imports* no utilizados, variables redundantes, código muerto, errores de estilo, complejidad innecesaria y diversas malas prácticas de programación en Python. Además, se empleó para mantener una estructura homogénea

⁷²https://sonarcloud.io/project/overview?id=lbr1014_TFG_RAG

⁷³<https://docs.astral.sh/ruff/>

en todo el proyecto, como por ejemplo, en los nombres de las funciones y variables. Esto ayudó a mejorar la legibilidad, consistencia y mantenibilidad del código, especialmente en un proyecto grande y modular como PythIA, donde coexistían múltiples componentes relacionados con Flask, el sistema RAG, el procesamiento documental y las tareas asíncronas.

Para mejorar la legibilidad también se añadieron comentarios y documentación y se realizó separación de responsabilidades.

Además del análisis estático, la calidad del código también se complementó con pruebas unitarias y pruebas funcionales desarrolladas durante distintas fases del proyecto. Estas pruebas permitieron verificar el correcto funcionamiento de los formularios, la gestión de usuarios, las validaciones, los servicios principales, la integración del sistema *RAG* y el procesamiento documental.

5.8. Despliegue

Uno de los aspectos más importantes durante el desarrollo de PythIA fue el diseño de un entorno de despliegue estable, reproducible y preparado para producción.

En las primeras fases del proyecto, la aplicación se ejecutaba en entornos locales de desarrollo utilizando el servidor integrado de Flask. Aunque esta aproximación era suficiente para validar prototipos y desarrollar funcionalidades, presentaba importantes limitaciones relacionadas con el rendimiento, la escalabilidad, la seguridad y la reproducibilidad del entorno.

Para resolver estos problemas se decidió utilizar Docker. Lo que permitió encapsular los componentes necesarios de la aplicación dentro de contenedores independientes, garantizando que el sistema se ejecute de manera consistente independientemente del entorno donde fuese desplegado.

La utilización de Docker aportó múltiples ventajas al proyecto, por ejemplo permitió aumentar la reproductibilidad del entorno, aislar los servicios y simplificar la gestión de dependencias. Mejorando la mantenibilidad y la portabilidad entre distintos entornos. Además, hizo que el despliegue fuese más fácil.

En las primeras versiones, Flask seguía ejecutándose utilizando su servidor de desarrollo integrado. Sin embargo, este servidor no está diseñado para entornos productivos, ya que no gestiona adecuadamente múltiples peticiones concurrentes ni ofrece suficiente robustez frente a cargas elevadas.

Para solucionar estos problemas se integró Gunicorn como servidor WSGI de producción. El cual se encarga de ejecutar múltiples workers de Flask de manera concurrente, mejorando significativamente el rendimiento y la estabilidad del sistema. Permitiendo gestionar mejor las peticiones simultáneas, el tiempo de espera, los procesos bloqueantes y la recuperación frente a fallos.

Otro aspecto importante fue la incorporación de Nginx como *proxy* inverso. Nginx se sitúa delante de Gunicorn y asume la gestión de conexiones HTTP, el balanceo interno de peticiones, la gestión de cabeceras, la optimización de rendimiento, la seguridad básica de acceso y se encarga de servir los archivos estáticos. Su utilización de manera conjunta con Gunicorn permitió separar las responsabilidades del sistema *web*, mejorando tanto el rendimiento como la seguridad de la aplicación.

La configuración de Docker presento alguna dificultad, por ejemplo, en la integración entre Flask, Qdrant y PostgreSQL, ya que la aplicación dependía simultáneamente de una base de datos relacional y otra vectorial. Lo que obligó a diseñar cuidadosamente la inicialización y configuración de los servicios para evitar errores durante el arranque del sistema.

Otro aspecto relevante fue la gestión de las variables sensibles y claves privadas. Para ello se utilizaron archivos de variables de entorno diferenciados, separando las configuraciones públicas de las credenciales privadas del sistema. Esto permitió mejorar la seguridad y facilitar la configuración en distintos entornos.

La arquitectura final del despliegue está dividida en varios contenedores especializados que se comunican entre sí mediante redes Docker internas. El sistema se divide principalmente en cinco servicios, un contenedor PostgreSQL para la persistencia de datos relacionales de la aplicación, un contenedor Qdrant encargado de la base de datos vectorial, un contenedor Ollama responsable de ejecutar los modelos de lenguaje y OCR multimodal utilizando aceleración por GPU, un contenedor principal con la aplicación Flask ejecutada mediante Gunicorn y un contenedor Nginx utilizado como *proxy* inverso y punto de entrada HTTPS del sistema. Es decir, la aplicación web no se comunica directamente con el exterior, sino que todas las peticiones pasan previamente por Nginx. Esto permite mejorar la seguridad y gestionar certificados TLS y balanceo de peticiones. Además, se incorporan volúmenes persistentes para almacenar datos de PostgreSQL, índices vectoriales de Qdrant, modelos descargados de Ollama y cachés de HuggingFace, evitando perder información al reiniciar los contenedores. La arquitectura también incluye comprobaciones automáticas de estado (*healthchecks*) y dependencias

entre servicios para asegurar que cada componente esté disponible antes de iniciar los demás.

5.9. Integración CI/CD

Durante el desarrollo también se incorporaron mecanismos de integración continua y automatización del flujo de desarrollo mediante GitHub Actions.

En las primeras fases del proyecto, todas las comprobaciones del sistema se realizaban manualmente. Cada modificación requería ejecutar localmente las pruebas, comprobar dependencias y verificar manualmente el correcto funcionamiento de la aplicación. Conforme el proyecto fue creciendo, este enfoque comenzó a resultar poco escalable y propenso a errores.

Para mejorar este proceso se decidió incorporar un *pipeline* CI/CD utilizando GitHub Actions. La integración se configuró para ejecutarse automáticamente tanto al realizar *commits* sobre la rama principal (*main*) como al abrir o actualizar *pull requests*. De esta forma cada cambio realizado en el repositorio desencadena las comprobaciones de calidad del sistema.

El *pipeline* CI/CD comienza clonando el repositorio y configurando el entorno Python utilizado por el proyecto. Luego instala las dependencias definidas para el entorno Docker de producción. Esto garantiza que las comprobaciones se realicen en un entorno lo más parecido posible al despliegue real. Una vez preparado el entorno, se ejecutan de manera automática los *tests* del proyecto mediante `unittest`. Además, durante la ejecución de las pruebas se calcula la cobertura de código utilizando `coverage.py`, generándose informes detallados sobre qué partes del sistema están siendo verificadas por los *tests* y cuales no. Tras la ejecución de las pruebas, GitHub Actions integra automáticamente el proyecto con SonarCloud para realizar el análisis estático del código.

6. Trabajos relacionados

En este apartado se analizan diferentes trabajos y proyectos que, aunque no constituyen modelos *RAG* idénticos al desarrollado en este trabajo, comparten principios fundamentales relacionados con el uso de *LLM*, recuperación de información, agentes inteligentes y gestión de contexto. Su estudio permite contextualizar el proyecto dentro del marco actual de aplicaciones basadas en *LLM* y comprender distintas aproximaciones al diseño de inteligencias artificiales.

6.1. TFG relacionados

***Retrieval-Augmented Generation* para la Extracción de Información de Documentos Inteligente**

En este TFG [17] se desarrolla un sistema RAG orientado al ámbito sanitario con el propósito de optimizar la recuperación y generación de información médica. El trabajo plantea una arquitectura basada en la recuperación y generación, que indexa documentación médica, crea *embeddings* y orquesta *prompts* para aportar contexto fiable al modelo generativo. Emplea técnicas de *chunking* con solapamiento y un enfoque *MultiQuery Retriever* para mejorar la cobertura y la precisión de la recuperación previa a la síntesis con un LLM. La solución se implementa sobre la API de *Azure OpenAI*, utilizando *LangChain* y *FAISS* como motor de búsqueda vectorial, e integra una interfaz web en *Flask* para simular consultas clínicas reales. Para la evaluación del sistema se emplea el *framework RAGAS*, valorando métricas de precisión, relevancia y coherencia de las respuestas generadas, demostrando mejoras significativas frente a enfoques puramente generativos.

6.2. Proyectos relacionados

LLM Twin Course: Building Your Production-Ready AI Replica

Este proyecto [11] propone la construcción de un asistente conversacional que actúa como el usuario. Su objetivo principal es crear un sistema capaz de responder, razonar y comunicarse tal y como lo haría un usuario concreto. Para ello debe adaptarse al conocimiento, las preferencias y el estilo de dicho usuario. A nivel técnico, el proyecto emplea una arquitectura modular que combina los modelos *LLM* para la generación de respuestas, con sistemas de memoria persistentes, recuperación de información contextual y orquestación de flujos conversacionales. Uno de los aspectos más relevantes es la integración de memoria externa, que permite almacenar conocimiento personalizado y recuperarlo de manera dinámica durante la conversación. Aunque no se presenta como un *RAG* clásico tiene el mismo objetivo, ampliar la memoria paramétrica del modelo mediante información recuperada.

Building Your Second Brain AI Assistant Using Agents, LLMs and RAG

Este proyecto [12] plantea la creación de un asistente para que organice, recupere y sintetice información personal o profesional. Para realizar esto emplea técnicas de agentes inteligentes y *RAG*. Su arquitectura combina sistemas de recuperación semántica basados en *embeddings*, orquestación mediante agentes, gestión de memoria persistente e integración directa con *LLM* para síntesis de información. Para implementar el modelo *RAG* la información se indexa, se recupera según la consulta del usuario y se incorpora al *prompt* aumentado del modelo. A este flujo se le añaden agentes que coordinan las tareas de búsqueda, filtrado y razonamiento para ampliar las capacidades del sistema.

ChatALL

Este proyecto [8] consiste en diseñar una herramienta que permite interactuar de manera simultánea con múltiples modelos de lenguaje (*LLM*). Su objetivo principal es permitir la comparación de respuestas entre distintos *LLM* desde una única interfaz. Su arquitectura se centra en la orquestación de múltiples *APIs* de modelos, en la gestión concurrente de respuestas y en el desarrollo de una interfaz unificada para la conversación. Aunque no amplía el conocimiento del modelo mediante el uso de *RAG* es relevante como

herramienta comparativa. Facilita evaluar el comportamiento de distintos *LLM* ante una misma consulta.

NotebookLM

NotebookLM⁷⁴ [21] [20] es una herramienta orientada a la interacción inteligente con colecciones de documentos proporcionados por el usuario. Permite cargar documentos, analizarlos y generar respuestas fundamentales en su contenido. Se basa en la indexación de documentos, la recuperación contextual y la generación asistida por *LLM*. Implementa un modelo *RAG* ya que responde utilizando el contenido recuperado. Esto reduce las alucinaciones y mejora la respuesta.

6.3. Análisis de las características de los proyectos

En la tabla 6.1 se muestra una comparativa entre las características de los distintos proyectos explicados previamente y las del modelo *RAG* desarrollado en el TFG.

⁷⁴<https://notebooklm.google.com/notebook/4696bf9f-51cb-42c0-8805-401c6286dbec>

Características	<i>LLM Twin Course</i>	<i>Second Brain AI Assistant</i>	<i>ChatAll</i>	<i>NotebookLM</i>	PythIA
Uso de <i>RAG</i>	Parcial	✓	×	✓	
Memoria externa	✓	✓	×	✓	
Multiagente	×	✓	×	×	
Comparación de modelos	×	×	✓	×	
Orientación documental	Media	Alta	×	Muy alta	
Enfoque principal	Réplica digital personalizada	Gestión inteligente del conocimiento	Comparación de <i>LLM</i>	Consulta documental guiada	

Tabla 6.1: Comparativa de las características de los proyectos.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Alejandro Alija. Técnicas *RAG*: cómo funcionan y ejemplos de casos de uso. <https://datos.gob.es/es/blog/tecnicas-rag-como-funcionan-y-ejemplos-de-casos-de-uso>, 2024. [Online; Accedido 6-Febrero-2026].
- [2] Inc. Cloudflare. ¿por qué el http no es seguro? | http vs. https. <https://www.cloudflare.com/es-es/learning/ssl/why-is-http-not-secure/>, 2026. [Online; Accedido 26-Febrero-2026].
- [3] Inc. Cloudflare. ¿qué es un proxy inverso? | servidores proxy explicados. <https://www.cloudflare.com/es-es/learning/cdn/glossary/reverse-proxy/>, 2026. [Online; Accedido 26-Febrero-2026].
- [4] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. pages 150–158, mar 2024. doi: 10.18653/v1/2024.eacl-demo.16. URL <https://aclanthology.org/2024.eacl-demo.16/>.
- [5] Pandora FMS. Qué es nat y cómo actúa en nuestra red. <https://pandorafms.com/es/it-topics/que-es-nat/>, 2005-2026. [Online; Accedido 26-Febrero-2026].
- [6] Gobierto. Qué es la contratación pública. <https://www.gobierto.es/blog/que-es-la-contratacion-publica>, 2024. [Online; Accedido 22-Febrero-2026].
- [7] Juan José Lozano Gómez. Tutorial de flask en español: Desarrollando una aplicación web en python. <https://j2logo.com/tutorial-flask-espanol/>, 2018-2025. [Online; Accedido 2-Diciembre-2025].

- [8] Peter Dave Hello. Chat with all ai bots concurrently, discover the best. <https://github.com/ai-shifu/ChatALL>, 2025. [Online; Accedido 21-Diciembre-2025].
- [9] Docker Inc. What is docker?. <https://docs.docker.com/get-started/docker-overview/>, 2013-2026. [Online; Accedido 3-Febrero-2026].
- [10] GitHub Inc. Acerca de github y git. <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>, 2025. [Online; Accedido 22-Septiembre-2025].
- [11] Paul Iusztin. Llm twin course: Building your production-ready ai replica. <https://github.com/decodingai-magazine/llm-twin-course?tab=readme-ov-file>, 2025. [Online; Accedido 9-Febrero-2026].
- [12] Paul Iusztin. Building your second brain ai assistant using agents, llms and rag. <https://github.com/decodingai-magazine/second-brain-ai-assistant-course>, 2025. [Online; Accedido 9-Febrero-2026].
- [13] Paul Iusztin and Maxime Labonne. *The LLM Engineer's Handbook*. Packt Publishing, October 2024. ISBN 9781836200079. [Consultado 30-Octubre-2025 a 27-Noviembre-2025].
- [14] Abhinav Kimothi. *A Simple Guide to Retrieval Augmented Generation*. Manning Publications, July 2025. ISBN 9781633435858. [Consultado 8-Septiembre-2025 a 26-Septiembre-2025].
- [15] Martijn Koster, G. Illyes, H. Zeller, and L. Sassman. Robots exclusion protocol (rfc 9309). <https://datatracker.ietf.org/doc/html/rfc9309>, 2022. [Online; Accedido 07-Octubre-2025].
- [16] Tom Krant and Alexandra Jonker. ¿qué es el envenenamiento de datos? <https://www.ibm.com/es-es/think/topics/data-poisoning>, 2024. [Online; Accedido 15-Mayo-2026].
- [17] Daniel Laparra Osca. Retrieval-augmented generation para la extracción de información de documentos inteligente. Trabajo fin de grado, Universitat Politècnica de València, Escuela Técnica Superior de Ingeniería de Telecomunicación, 2024. URL <https://riunet.upv.es/server/api/core/bitstreams/c775be7d-8724-476b-b0a4-73e055d03d50/content>. [Accedido 02-Octubre-2025].

- [18] Lofred Madzou. What is the rag triad? <https://truera.com/ai-quality-education/generative-ai-rags/what-is-the-rag-triad/>. [Online; Accedido 24-Septiembre-2025].
- [19] Julia Martins. ¿qué es la metodología kanban y cómo funciona?. <https://asana.com/es/resources/what-is-kanban>, 2026. [Online; Accedido 27-Febrero-2026].
- [20] Emergent Mind. Notebooklm: Document-grounded ai by google. <https://www.emergentmind.com/topics/notebooklm>, 2025. [Online; Accedido 10-Febrero-2026].
- [21] Emergent Mind. Notebooklm enterprise. <https://docs.cloud.google.com/gemini/enterprise/notebooklm-enterprise/docs/overview?hl=es>, 2026. [Online; Accedido 10-Febrero-2026].
- [22] Columbia University Mailman School of Public Health. Web scraping. <https://www.publichealth.columbia.edu/research/population-health-methods/web-scraping>, -. [Online; Accedido 28-Septiembre-2025].
- [23] W3C Working Group on Browser Testing and Tools. Webdriver — w3c working draft (level 2). <https://www.w3.org/TR/webdriver2/>, 2025. [Online; Accedido 15-Octubre-2025].
- [24] Orange. Diferencias entre ip pública e ip privada. <https://ayuda.orange.es/particulares/internet-y-wi-fi/configuracion-e-instalacion/6860-diferencias-entre-ip-publica-e-ip-privada>, 2026. [Online; Accedido 26-Febrero-2026].
- [25] Selenium Project. Selenium documentation. <https://www.selenium.dev/documentation/>, 2025. [Online; Accedido 10-Octubre-2025].
- [26] Ragas. Ragas documentation. <https://docs.ragas.io/en/stable/>, 2025. [Online; Accedido 05-Octubre-2025].
- [27] Megan Rogge and microsoft. Visual studio code - open source (code – oss). <https://github.com/microsoft/vscode>. [Online; Accedido 27-Febrero-2026].
- [28] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. Ares: An automated evaluation framework for retrieval-augmented generation systems. 2024. URL <https://arxiv.org/abs/2311.09476>. [Online; Accedido 05-Octubre-2025].

- [29] Josh Schneider and Ian Smalley. ¿qué es un entorno de desarrollo integrado (ide)?. <https://www.ibm.com/es-es/think/topics/integrated-development-environment>, 2018-2026. [Online; Accedido 27-Febrero-2026].
- [30] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>, 2020. [Online; Accedido 22-Septiembre-2025].
- [31] Q2B Studio. Por qué «lost in the middle» rompe la mayoría de los sistemas rag. <https://www.q2bstudio.com/nuestro-blog/429461/descubre-por-que-lost-in-the-middle-puede-causar-problemas-en-los-sistemas-rag-y-como-evitarlos>, 2026. [Online; Accedido 15-Mayo-2026].
- [32] Microsoft Playwright Team. Playwright documentation: Actionability. <https://playwright.dev/docs/actionability>, 2025. [Online; Accedido 10-Octubre-2025].
- [33] Joseph Wright. Tex on windows: Miktex or tex live? *TUGboat*, 33(1):53, 2012. URL <https://www.tug.org/TUGboat/tb33-1/tb103wright.pdf>. [Online; Accedido 22-Septiembre-2025].
- [34] WSGI.org. Wsgi. <https://wsgi.readthedocs.io/en/latest/>, 2016-2026. [Online; Accedido 25-Febrero-2026].
- [35] Zilliz. Qdrant vs. pgvector. <https://zilliz.com/comparison/qdrant-vs-pgvector>, 2025. [Online; Accedido 10-Noviembre-2025].